

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

OmniAI Gateway

Relatório Final de Projeto

Guilherme Coutinho

Nº 50467

A50467@alunos.isel.pt

André Nunes

Nº 51766

A51766@alunos.isel.pt

Orientador: Pedro Félix

pedro.felix@isel.pt

Engenharia Informática e de Computadores

27 de maio de 2026

Resumo

A ausência de um protocolo comum na comunicação com Modelos de Linguagem de Grande Escala (LLMs) obriga as aplicações a integrarem diferentes interfaces de acesso e implementações particulares para a invocação de ferramentas externas. Para resolver esta fragmentação tecnológica, este projeto propõe o desenvolvimento do **OmniAI SDK**, um *Software Development Kit* construído em Kotlin Multiplatform (KMP). Baseado nos princípios da Arquitetura Hexagonal (*Ports and Adapters*), o SDK isola a lógica de domínio e suporta a comunicação bidirecional compatível com os padrões das APIs de referência, nomeadamente OpenAI, Anthropic e Gemini. O núcleo do sistema implementa uma *pipeline* extensível e configurável, inspirada no modelo de filtros de Servlets, que executa *interceptors* pré-construídos para autorização, autenticação, métricas, *rate limiting*, *circuit breaker* e *fallback*. Adicionalmente, o SDK prepara um módulo MCP Broker para evoluir a gestão de ferramentas externas de forma independente dos adapters de inferência. Distribuído de forma multiplataforma para ambientes JVM, via Maven, e Node.js/TypeScript, via NPM, o SDK foi validado através da construção da **OmniAI Gateway**, um POC que consome a biblioteca para demonstrar a criação de uma Gateway real. A avaliação demonstra que a solução minimiza o acoplamento tecnológico e o esforço de integração, garantindo resiliência, escalabilidade e centralização na gestão de modelos de IA.

Abstract

The lack of a common protocol for communicating with Large Language Models (LLMs) forces applications to integrate different access interfaces and specific implementations for invoking external tools. To address this technological fragmentation, this project proposes the development of the OmniAI SDK, a Software Development Kit built on Kotlin Multiplatform (KMP). Based on the principles of the Hexagonal Architecture (Ports and Adapters), the SDK isolates the domain logic and supports bidirectional communication compatible with the standards of reference APIs, namely OpenAI, Anthropic, and Gemini. The system's core implements an extensible and configurable pipeline, inspired by the Servlet filter model, which executes pre-built interceptors for authorization, authentication, metrics, rate limiting, circuit breaker, and fallback. Additionally, the SDK prepares an MCP Broker module to evolve external tool management independently from inference adapters. Distributed across multiple platforms for JVM environments via Maven and Node.js/TypeScript via NPM, the SDK was validated through the development of the OmniAI Gateway, a proof-of-concept application that leverages the library to instantiate a real Gateway. The evaluation demonstrates that the solution minimizes technological coupling and integration effort, ensuring resilience, scalability, and centralization in AI model management.

Índice

1	Introdução	4
1.1	Contexto	4
1.2	Motivação e Definição do Problema	5
1.3	Objetivos e Proposta de Solução	6
1.4	Estrutura do Documento	7
2	Estudo Prévio	8
2.1	Análise das APIs de Inferência	8
2.2	Model Context Protocol (MCP)	11
3	Modelação do Domínio Comum	13
3.1	Visão Global do Domínio	13
3.2	Estruturas Internas de Pedido e Resposta	14
3.3	TypedMap e Extensibilidade do Domínio	15
4	Arquitetura e Desenho do Sistema	17
4.1	Arquitetura Top-Level	17
4.2	Arquitetura Hexagonal (Ports & Adapters)	18
4.2.1	Contracts	19
4.2.2	Inbounds	19
4.2.3	Outbounds	19
4.2.4	Dispatcher	20
4.3	Ecossistema do SDK	20
5	Implementação e Evolução Técnica	22
5.1	Stack Tecnológica e Metodologias	22
5.2	A Primeira Iteração da Gateway	22
5.3	Decisões de Desenho e Alternativas Descartadas	23
5.4	Evolução Arquitetural: De <i>Inference Service</i> a <i>Dispatcher</i>	24
5.5	Evolução para a Pipeline de Interceptors	24
5.6	Arquitetura Interna dos Interceptors	26
5.6.1	Interceptor de Telemetria e Observabilidade	27
5.6.2	Interceptor de Autenticação	29
5.6.3	Interceptor de Autorização	31

5.6.4	Interceptor de Rate Limiting	32
5.6.5	Interceptor de Circuit Breaker	33
5.6.6	Interceptor de Fallback	33
5.6.7	Interceptor de Logging e Tracing	34
5.7	MCP Broker e Ferramentas Externas	34
5.8	Distribuição Multiplataforma	35
6	Provas de Conceito (PoCs) e Validação	37
6.1	Validação do POC OmniAI Gateway	37
6.2	Validação com REST Client	38
6.3	Validação de Segurança com Logto	38
6.4	Validação de Observabilidade	38
6.5	Validação com Aplicações Cliente	39
6.6	Limites da Validação	39
7	Conclusões e Trabalho Futuro	40
7.1	Conclusões	40
7.2	Melhorias Futuras e Escalabilidade	40
A	Histórico de Planeamento e Riscos	41
A.1	Riscos	41
A.2	Planeamento	43

Lista de Figuras

1.1	Acoplamento direto entre cliente e um fornecedor de inferência	5
1.2	Multiplicação de conexões com chaves de API descentralizadas	5
1.3	Cliente decide trocar de fornecedor	5
1.4	Centralização com OmniAI Gateway	6
2.1	Normalização dos contratos de inferência para um domínio comum	11
3.1	Fluxo de normalização entre contratos externos e domínio comum	13
3.2	Estrutura interna principal de <code>CommonRequest</code>	14
3.3	Estruturas de resposta final e eventos progressivos	14
3.4	Utilização da <code>TypedMap</code> para transportar atributos extensíveis	15
4.1	Arquitetura top-level: POC OmniAI Gateway a instanciar e demonstrar o OmniAI SDK	17
4.2	Visão da integração entre Ktor, inbounds, pipeline, interceptors, outbounds e HTTP client no OmniAI SDK	18
5.1	Visão conceptual da pipeline de interceptors	25
5.2	Fluxo interno de processamento e lógica de observabilidade para pedidos unários e streaming	28
5.3	Observabilidade e métricas operacionais da OmniAI Gateway	29
5.4	Sequência OIDC: descoberta, JWKS e cache de chaves	30
5.5	Fluxo interno do interceptor de autenticação	31
5.6	Arquitetura do interceptor de autorização: separação entre PEP e PDP	32
5.7	Representação do ciclo de <i>tool calling</i> e interação com ferramentas externas	35
6.1	Cenários de validação do POC OmniAI Gateway	37

Capítulo 1

Introdução

1.1 Contexto

Os Modelos de Linguagem de Grande Escala (*Large Language Models – LLMs*) [1] tornaram-se componentes relevantes no desenvolvimento de aplicações baseadas em inteligência artificial. Um LLM é um modelo treinado com grandes volumes de texto, capaz de interpretar linguagem natural, gerar respostas e apoiar tarefas como sumarização, classificação, programação assistida, pesquisa semântica e interação com sistemas externos.

É possível distinguir duas fases principais no ciclo de vida de um modelo: treino e inferência. O treino corresponde ao processo de aprendizagem a partir de grandes conjuntos de dados, exigindo elevados recursos computacionais. A inferência corresponde à utilização do modelo já treinado para gerar respostas a novos pedidos. Atualmente, a maioria das aplicações integra LLMs através de APIs de inferência disponibilizadas por fornecedores externos.

A comunicação com estas APIs é normalmente realizada em JSON, tanto nos pedidos como nas respostas. Antes de serem processados pelos modelos, os textos são convertidos em *tokens*, pequenas unidades de informação que permitem ao modelo representar e manipular sequências de entrada e saída [2]. Como a resposta é gerada de forma incremental, muitas APIs suportam *streaming*, permitindo enviar fragmentos da resposta ao cliente à medida que são produzidos.

Existem vários fornecedores relevantes, incluindo OpenAI [3], Anthropic [4], Google Gemini [5] e Groq [6]. Embora todos disponibilizem APIs HTTP e estruturas JSON, cada fornecedor define contratos próprios, campos específicos, regras de autenticação e modelos distintos para *tool calling*. A integração direta com vários fornecedores obriga, por isso, as aplicações a conhecerem detalhes internos de cada API.

Ao mesmo tempo, o uso de LLMs evoluiu de interações simples para sistemas baseados em agentes (*agentic workflows*) [7]. Nestes cenários, o modelo não se limita a devolver texto; pode decidir invocar ferramentas externas, interpretar resultados intermédios e produzir uma resposta final com base nesses dados. O *Model Context Protocol* (MCP) surgiu como proposta de normalização para a comunicação entre modelos, aplicações e ferramentas externas [8].

1.2 Motivação e Definição do Problema

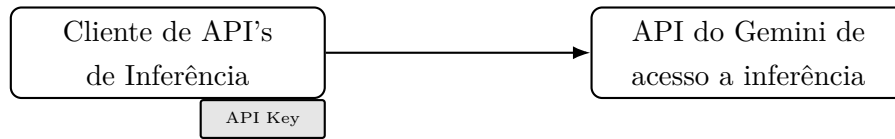


Figura 1.1: Acoplamento direto entre cliente e um fornecedor de inferência

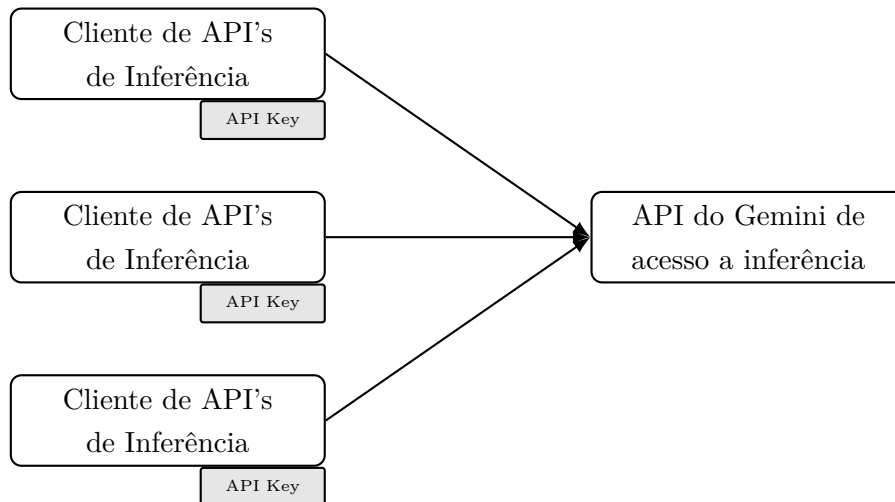


Figura 1.2: Multiplicação de conexões com chaves de API descentralizadas

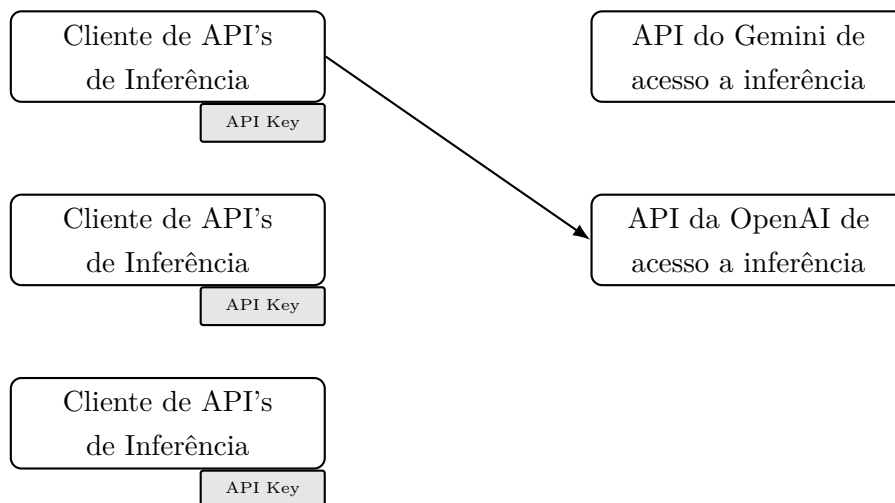


Figura 1.3: Cliente decide trocar de fornecedor

As Figuras 1.1, 1.2 e 1.3 ilustram o acoplamento direto entre clientes e fornecedores de inferência, bem como a dificuldade que os clientes tem em trocar de fornecedor devido a todo acoplamento que existe

A existência de múltiplos fornecedores de LLMs e a ausência de um protocolo comum implicam diferentes interfaces de acesso aos modelos, estruturas específicas de pedidos e respostas,

modelos distintos de contabilização de tokens, custos variáveis e implementações particulares para invocação de funções externas (ver Figuras 1.1–1.3). Por exemplo, a API OpenAI organiza a interação em torno de chat completions e tool calls; a Anthropic utiliza uma arquitetura baseada em messages e blocos de conteúdo; e a Gemini estrutura a geração através de contents, parts, function declarations e configurações de geração próprias.

Estas diferenças criam acoplamento entre as aplicações cliente e os fornecedores. Sempre que uma aplicação pretende trocar de modelo, suportar mais do que um fornecedor, adicionar um mecanismo de *fallback* ou integrar ferramentas externas, torna-se necessário duplicar lógica de tradução, autenticação, tratamento de erros e interpretação de respostas. Esta duplicação aumenta a complexidade e dificulta a evolução do sistema, um problema recorrente em sistemas que combinam lógica de aplicação com integrações externas especializadas [9, 10].

Para além da fragmentação dos contratos, existem preocupações operacionais e de segurança. Uma aplicação que comunica diretamente com vários fornecedores precisa de gerir chaves de API, quotas, limites de utilização, métricas, auditoria, políticas de acesso e proteção contra falhas de terceiros. Sem uma camada intermédia, estas preocupações ficam espalhadas por vários pontos do sistema, tornando mais difícil aplicar regras consistentes de autenticação, autorização, observabilidade e resiliência.

O problema central deste projeto é, assim, a ausência de uma camada de abstração capaz de separar as aplicações cliente das particularidades de cada fornecedor de LLMs, mantendo simultaneamente suporte para segurança, monitorização, *tool calling* e integração dinâmica com ferramentas externas.

1.3 Objetivos e Proposta de Solução

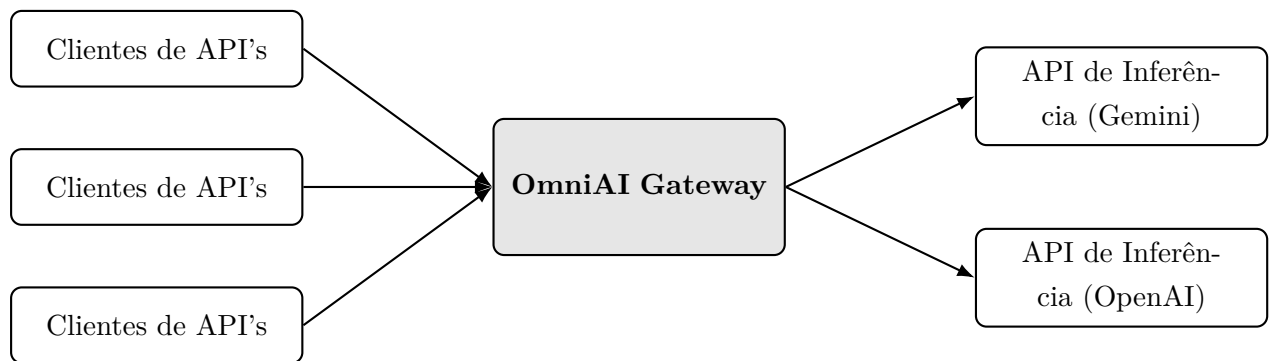


Figura 1.4: Centralização com OmniAI Gateway

A solução proposta evoluiu para a construção de um **OmniAI SDK**, isto é, uma biblioteca reutilizável que permite instanciar uma **OmniAI Gateway**. A Gateway funciona como ponto de entrada único para aplicações cliente, centralizando o acesso a APIs de inferência, ferramentas externas e mecanismos transversais de segurança e operação. Esta abordagem aproxima-se do padrão de API Gateway [11] e, no contexto específico de modelos de IA, do padrão de AI Gateway enquanto camada centralizada de acesso a modelos [12, 13]. Embora existam soluções

relacionadas, como LiteLLM e OpenRouter [14, 15], o foco deste projeto está na construção de um SDK extensível, configurável e validado por uma Gateway POC própria.

O SDK foi desenvolvido em Kotlin Multiplatform, permitindo partilhar o núcleo de domínio entre targets diferentes e preparar a distribuição para JVM e JavaScript/Node.js [16]. Esta decisão torna possível disponibilizar a biblioteca em ecossistemas distintos: via Maven para aplicações JVM/Kotlin e via NPM para ambientes Node.js/TypeScript.

A arquitetura do SDK foi organizada segundo os princípios da Arquitetura Hexagonal, também conhecida como *Ports and Adapters* [17]. O domínio comum fica isolado de contratos externos, enquanto os *inbound adapters* traduzem pedidos de clientes compatíveis com OpenAI, Anthropic ou Gemini para o modelo comum, e os *outbound adapters* traduzem o modelo comum para chamadas reais aos fornecedores.

Para tornar a solução configurável e escalável, foi criada uma *pipeline* interna de *interceptors*, inspirada no modelo de filtros de Servlets [18]. Esta *pipeline* permite executar comportamentos transversais antes e depois da chamada ao fornecedor, incluindo autenticação, autorização, métricas, *rate limiting*, *circuit breaker*, *fallback* e *logging*. Por fim, o estudo do MCP fundamenta a evolução de um broker de ferramentas separado da lógica de inferência.

1.4 Estrutura do Documento

Este documento encontra-se organizado de forma progressiva, partindo do problema de integração e avançando até à validação da solução construída. O Capítulo 2 apresenta apenas o estudo prévio: primeiro a análise comparativa das APIs de inferência e depois o estudo do *Model Context Protocol*. O Capítulo 3 isola a modelação do domínio comum, explicando como os conceitos observados no estudo foram transformados em estruturas como `CommonRequest`, `CommonResponse`, `CommonResponseEvent`, `CommonTool` e `TypedMap`. O Capítulo 4 descreve a arquitetura do SDK e do POC, incluindo a visão top-level, a Arquitetura Hexagonal, os módulos, os *contracts*, os *inbounds*, os *outbounds*, o *dispatcher*, a DSL e o cliente HTTP. O Capítulo 5 detalha a implementação técnica e a evolução do sistema, com foco na separação entre SDK e OmniAI Gateway, na *pipeline* de *interceptors*, nos mecanismos de segurança, observabilidade e resiliência, no MCP Broker e na distribuição multiplataforma. O Capítulo 6 apresenta as provas de conceito efetivamente realizadas no POC OmniAI Gateway. O Capítulo 7 sintetiza as conclusões, limitações e linhas de evolução futura. O Capítulo 8 regista o histórico de planeamento e os principais riscos considerados durante o desenvolvimento.

Capítulo 2

Estudo Prévio

2.1 Análise das APIs de Inferência

O estudo inicial comparou APIs de inferência compatíveis com os fornecedores OpenAI, Anthropic e Gemini, bem como execuções locais e compatíveis através de Ollama e de endpoints OpenAI-like, como Groq. Esta análise não teve apenas um papel descritivo: foi a base empírica para identificar os conceitos que podiam ser normalizados no SDK e os pontos onde a normalização teria de preservar extensões específicas de cada fornecedor. Para esse efeito foram executados e documentados pedidos de geração simples, pedidos com *streaming*, pedidos com *tool calling*, respostas com contabilização de tokens e casos de configuração de geração.

A comparação mostrou que as APIs partilham um núcleo conceptual comum, mas expõem esse núcleo através de modelos de dados diferentes. A OpenAI organiza a interação em torno de `chat/completions`, com `messages`, `choices`, `tool_calls` e eventos incrementais baseados em `delta` [3]. A Anthropic usa uma API de `messages`, onde o `system prompt` está no nível de topo, o campo `max_tokens` é obrigatório, e o conteúdo da resposta é uma lista de blocos como `text`, `tool_use` ou `thinking` [4]. A Gemini modela o pedido como `contents` compostos por `parts`, agrupa parâmetros em `generationConfig`, representa ferramentas por `functionDeclarations` e introduz conceitos próprios como *thought signatures* e `thinkingConfig` [5].

Conceito	Semelhança observada	Diferenças relevantes	Modelo comum pretendido
Modelo invocado	As três APIs precisam de identificar o modelo a usar.	OpenAI e Anthropic usam <code>model</code> no corpo; Gemini coloca o modelo no URI e, por vezes, também no cliente.	<code>provider + model</code> , permitindo escolher adapter e preservar a identidade do modelo.
Histórico conversacional	Todas representam uma conversa com papéis e conteúdo.	OpenAI e Anthropic usam <code>messages</code> ; Gemini usa <code>contents</code> e <code>parts</code> ; Anthropic separa o <code>system</code> .	<code>CommonRequestMessage</code> , com papel comum e lista de partes de conteúdo.
Instruções de sistema	Todas suportam instruções globais para orientar o modelo.	OpenAI pode usar mensagem <code>system</code> ; Anthropic usa campo de topo <code>system</code> ; Gemini usa <code>system_instruction</code> .	<code>SystemPrompt</code> , separado do histórico para permitir tradução correta.
Parâmetros de geração	Temperatura, limite de tokens e opções de amostragem aparecem nos três ecossistemas.	Nomes e agrupamentos variam: campos soltos em OpenAI/Anthropic, <code>generationConfig</code> em Gemini, e opções exclusivas por fornecedor.	<code>CommonGenerationConfig</code> para o subconjunto comum e <code>providerOptions</code> para extensões.
Conteúdo multimodal e estruturado	As APIs evoluem para conteúdo composto, não apenas texto simples.	Anthropic e Gemini modelam blocos naturalmente; OpenAI depende do contrato usado; tool calling introduz partes específicas.	<code>RequestContentPart</code> e <code>ResponseContentPart</code> .
Utilização de tokens	Todas expõem algum tipo de contabilização de utilização.	OpenAI usa <code>prompt_tokens/ completion_tokens</code> ; Anthropic usa <code>input_tokens/ output_tokens</code> ; Gemini usa <code>usageMetadata</code> .	<code>CommonUsage</code> , com tokens de entrada, saída e total quando disponíveis.

Tabela 2.1: Mapeamento entre conceitos das APIs de inferência e o domínio comum

O caso de *tool calling* foi especialmente relevante, porque é onde as APIs mais divergem apesar de exprimirem uma intenção semelhante: permitir que o modelo peça a execução de uma função externa. Na OpenAI, a ferramenta é descrita em `tools[]` e a resposta devolve `tool_calls` com argumentos serializados como JSON em string. Na Anthropic, a ferramenta é descrita com `name`, `description` e `input_schema`; a resposta inclui blocos `tool_use`, e o resultado da execução regressa ao modelo como bloco `tool_result`. Na Gemini, as ferramentas são `functionDeclarations`, a chamada surge em `functionCall`, e o resultado é enviado como `functionResponse`.

Aspeto	API	Estrutura observada	Impacto no domínio comum
Declaração de ferramenta	OpenAI	<code>tools[].function.parameters.</code>	Normalização em <code>CommonTool.parametersSchema.</code>
Declaração de ferramenta	Anthropic	<code>tools[].input_schema.</code>	Mesmo conceito, nome de campo diferente.
Declaração de ferramenta	Gemini	<code>functionDeclarations[].parameters.</code>	Necessidade de traduzir declarações Gemini para a mesma estrutura comum.
Escolha de ferramenta	OpenAI	<code>tool_choice</code> com valores como <code>auto</code> , <code>none</code> ou ferramenta específica.	Representação em <code>ToolChoice.</code>
Escolha de ferramenta	Anthropic	<code>tool_choice</code> com semântica semelhante, incluindo <code>auto</code> , <code>any</code> e ferramenta específica.	O domínio preserva intenção, não o nome exato do campo.
Escolha de ferramenta	Gemini	<code>functionCallingConfig</code> dentro de <code>toolConfig.</code>	Conversão para a mesma decisão comum sobre uso de ferramenta.
Chamada pedida pelo modelo	OpenAI	<code>message.tool_calls[]</code> com <code>id</code> , <code>name</code> e <code>arguments.</code>	Mapeamento para <code>ToolCallPart.</code>
Chamada pedida pelo modelo	Anthropic	Bloco <code>tool_use</code> com <code>id</code> , <code>name</code> e <code>input.</code>	A chamada passa a ser uma parte estruturada da resposta.
Chamada pedida pelo modelo	Gemini	<code>functionCall</code> com <code>name</code> , <code>args</code> e, quando disponível, <code>id.</code>	O domínio não assume que todos os fornecedores expõem os mesmos identificadores.
Resultado de ferramenta	OpenAI	Mensagem com <code>role=tool</code> e <code>tool_call_id.</code>	Mapeamento para <code>ToolResultPart.</code>
Resultado de ferramenta	Anthropic	Bloco <code>tool_result.</code>	Resultado representado como conteúdo estruturado.
Resultado de ferramenta	Gemini	<code>functionResponse.</code>	Resultado traduzido para o mesmo conceito comum.
Eventos de <i>streaming</i>	Todas	OpenAI usa <code>delta</code> ; Anthropic usa eventos nomeados; Gemini envia respostas parciais por SSE.	Representação em <code>CommonResponseEvent</code> , separada de <code>CommonResponse.</code>

Tabela 2.2: Comparação do ciclo de *tool calling* e *streaming*

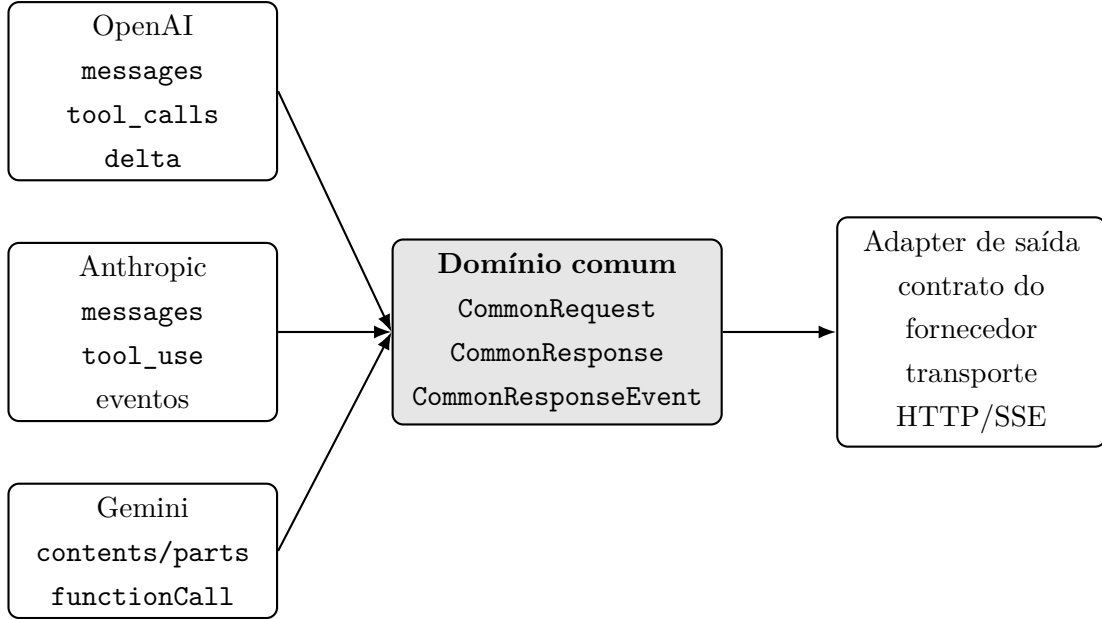


Figura 2.1: Normalização dos contratos de inferência para um domínio comum

A conclusão principal deste estudo é que a construção do domínio comum não podia ser feita por intuição nem por uma tradução superficial de nomes de campos. Foi necessário observar pedidos e respostas reais, perceber como cada API representa mensagens, ferramentas, eventos e erros, e decidir que conceitos pertenciam ao núcleo comum e que conceitos deveriam permanecer como extensões específicas. A Tabela 2.1 mostra essa distinção: campos recorrentes como mensagens, modelo, parâmetros de geração e utilização de tokens justificam estruturas comuns; já elementos como *thought signatures*, parâmetros avançados de raciocínio ou metadados específicos exigem um mecanismo extensível para não se perder informação.

2.2 Model Context Protocol (MCP)

O *Model Context Protocol* define uma arquitetura cliente-servidor para disponibilizar contexto a aplicações baseadas em modelos. Na terminologia do protocolo, um *host* é a aplicação de IA que inicia e coordena ligações, os *clients* mantêm ligações 1:1 com servidores, e os *servers* expõem capacidades como ferramentas, recursos e *prompts* através de mensagens JSON-RPC e transportes normalizados [8, 19]. Esta separação é relevante porque transforma integrações externas em unidades descobertas e invocadas através de um protocolo comum, em vez de as embutir diretamente na API de inferência de cada fornecedor.

O estudo do MCP foi realizado em paralelo com a análise das APIs de inferência, mas com um objetivo diferente. Enquanto a análise das APIs procurou compreender pedidos e respostas de modelos, o estudo do MCP procurou compreender como ferramentas externas podem ser descritas, listadas e executadas de forma independente do fornecedor de LLM. Os conceitos centrais observados foram:

- **Tools:** funções ou ações executáveis por um servidor MCP, descritas por nome, descrição e esquema de entrada.

- **Resources:** dados ou documentos que podem ser descobertos e lidos por clientes MCP, normalmente identificados por URIs estáveis.
- **Prompts:** modelos reutilizáveis de interação, que podem encapsular instruções, argumentos e mensagens estruturadas.
- **Transports:** mecanismos de comunicação como `stdio`, SSE, HTTP com *streaming* e WebSocket, dependendo do ambiente de execução.

Para ganhar contexto prático, foi criado um projeto experimental de pequena dimensão, usado como “projeto brinquedo”, onde foram implementados servidores MCP e clientes MCP com a biblioteca Kotlin MCP. Esse protótipo permitiu validar o ciclo básico de criação de um servidor, registo de capacidades, ligação a partir de um cliente e troca de mensagens. A utilização da biblioteca oficial `io.modelcontextprotocol:kotlin-sdk` também mostrou que a abordagem é compatível com Kotlin Multiplatform e que existem APIs próprias para construir clientes, servidores, ferramentas, recursos, *prompts* e transportes [20].

Esta análise ajudou a distinguir dois problemas que, à primeira vista, podiam parecer iguais. O primeiro problema é a normalização de contratos de inferência, resolvido no SDK através de *contracts*, *translators*, *inbounds*, *outbounds* e domínio comum. O segundo problema é a gestão de ferramentas externas, onde o MCP oferece uma forma padronizada de descrever e executar capacidades. No final deste estudo, tornou-se natural desenhar um módulo de MCP Broker: não como ponto de partida do Capítulo 2, mas como consequência da necessidade de intermediar ferramentas externas sem acoplar cada adapter de inferência a cada ferramenta disponível.

Capítulo 3

Modelação do Domínio Comum

3.1 Visão Global do Domínio

A modelação do domínio comum do OmniAI SDK foi definida a partir dos padrões observados no estudo prévio. A ideia central foi criar um modelo de dados suficientemente expressivo para representar pedidos, respostas e eventos dos fornecedores estudados, mas sem copiar diretamente a estrutura de nenhum deles. Assim, o domínio funciona como um ponto intermédio estável: os adapters de entrada convertem contratos externos para o domínio; os adapters de saída convertem o domínio para o contrato do fornecedor real.

O pedido comum é representado por **CommonRequest**. Esta estrutura contém o fornecedor pretendido, o modelo, a lista de mensagens, o *system prompt*, a configuração de geração, as ferramentas disponíveis, a escolha de ferramenta, a indicação de resposta JSON e um campo extensível para opções específicas do fornecedor. O campo **provider** é importante porque o SDK suporta contratos inbound compatíveis com um fornecedor e execução outbound noutro contexto configurado; o campo **model** permite distinguir múltiplos modelos dentro do mesmo fornecedor.

A resposta síncrona é representada por **CommonResponse**. Esta estrutura inclui fornecedor, identificador, modelo, escolhas, mensagem de resposta, razão de terminação e utilização de tokens. Em paralelo, as respostas incrementais são representadas por **CommonResponseEvent**. Esta separação evita forçar um fluxo de *streaming* a parecer uma resposta final e permite modelar eventos que não existem numa resposta única, como início de escolha, fragmento de texto, início de chamada de ferramenta, fragmento de argumentos, utilização reportada e conclusão.

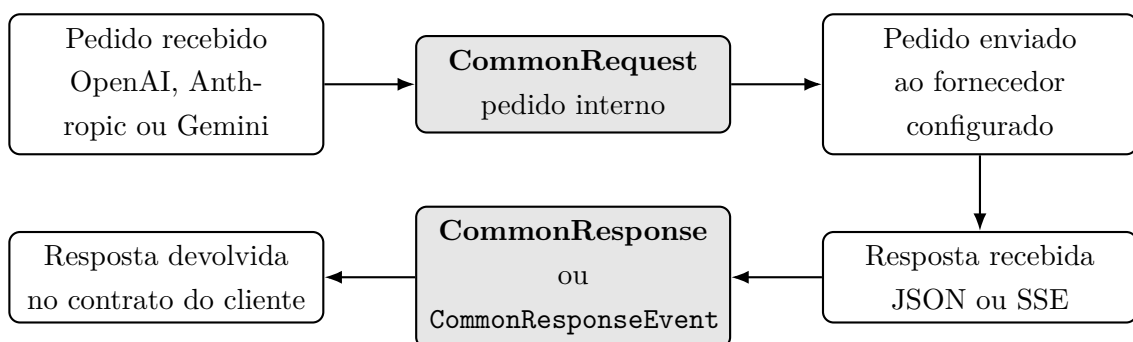


Figura 3.1: Fluxo de normalização entre contratos externos e domínio comum

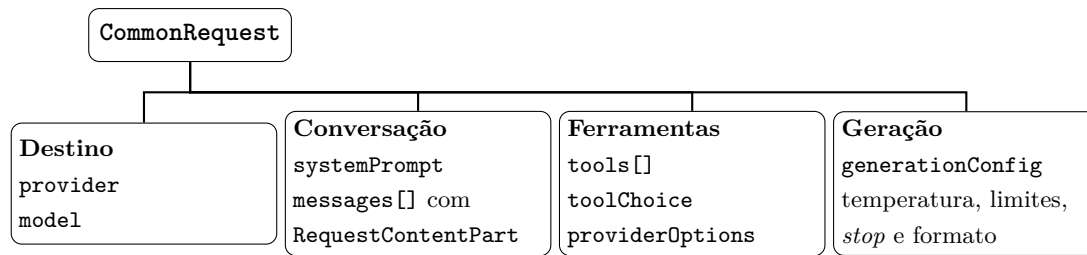


Figura 3.2: Estrutura interna principal de `CommonRequest`

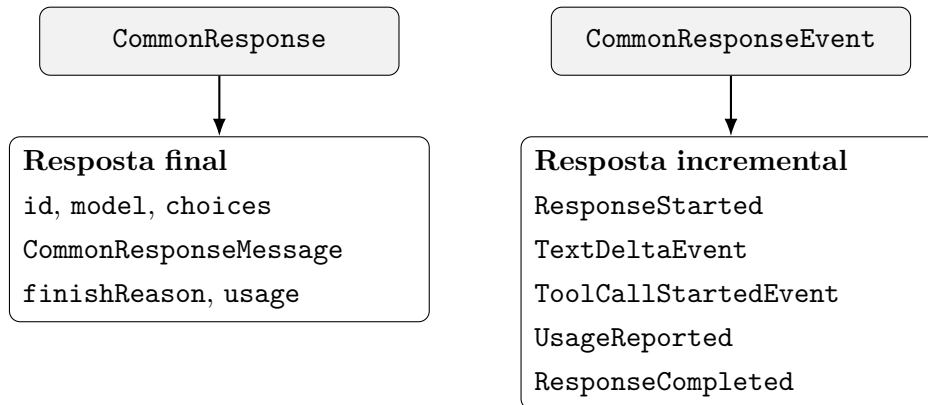


Figura 3.3: Estruturas de resposta final e eventos progressivos

3.2 Estruturas Internas de Pedido e Resposta

As mensagens do pedido são representadas por `CommonRequestMessage`, que combina um `CommonRole` com uma lista de `RequestContentPart`. Esta decisão evita assumir que uma mensagem é sempre uma string simples. No estudo das APIs, essa suposição mostrou-se insuficiente: a Anthropic e a Gemini trabalham naturalmente com blocos de conteúdo, e o *tool calling* introduz partes que representam chamadas e resultados de ferramentas. Por esse motivo, o domínio contém tipos como `TextPart`, `JsonPart`, `ToolCallPart` e `ToolResultPart`.

Na resposta, a estrutura `CommonChoice` preserva o índice da escolha e a respetiva `CommonResponseMessage`. Isto permite representar APIs que devolvem várias alternativas, mantendo compatibilidade com fornecedores que devolvem apenas uma resposta. A razão de terminação é normalizada em `FinishReason`, com valores como `STOP`, `LENGTH`, `TOOL_CALL`, `CONTENT_FILTER` e `OTHER`. Esta normalização é deliberadamente pequena: cobre os casos que têm significado transversal e deixa detalhes particulares em campos de extensão.

Estrutura de domínio	Responsabilidade	Origem no estudo prévio
<code>CommonRequest</code>	Representar o pedido de inferência de forma independente do contrato recebido.	Campos recorrentes: modelo, mensagens, parâmetros, ferramentas e formato de resposta.
<code>CommonRequestMessage</code>	Preservar o papel da mensagem e permitir conteúdo composto.	Diferença entre <code>messages</code> , <code>contents</code> e blocos de conteúdo.
<code>CommonTool</code>	Descrever uma ferramenta por nome, descrição e esquema de parâmetros.	Mapeamento entre <code>tools</code> , <code>input_schema</code> e <code>functionDeclarations</code> .
<code>ToolChoice</code>	Indicar se o modelo pode, não pode, deve ou tem de usar uma ferramenta específica.	Diferenças entre <code>tool_choice</code> e configurações Gemini de chamada de função.
<code>CommonResponse</code>	Representar a resposta final e a utilização de tokens.	Diferenças entre <code>choices</code> , blocos <code>content</code> e <code>candidates</code> .
<code>CommonResponseEvent</code>	Representar respostas progressivas e eventos de ferramenta em <i>streaming</i> .	SSE e eventos incrementais observados em OpenAI, Anthropic e Gemini.

Tabela 3.1: Principais estruturas do domínio comum

3.3 TypedMap e Extensibilidade do Domínio

Nem todos os dados relevantes pertencem ao modelo mínimo de inferência. Algumas informações são específicas do fornecedor, como parâmetros avançados da Gemini; outras pertencem ao contexto operacional, como metadados de autenticação, IP do cliente, cabeçalhos HTTP ou atributos necessários para observabilidade e autorização. Para estes casos, o SDK introduz a estrutura `TypedMap`.

A `TypedMap` funciona como um mapa de atributos tipado por chave. Cada `AttributeKey<T>` associa um nome lógico a um tipo esperado, permitindo guardar e recuperar valores sem depender de conversões frágeis de strings. Esta abordagem é mais segura do que um `Map<String, Any?>` puro, porque cada chave transporta também a expectativa de tipo. Ao mesmo tempo, é mais flexível do que adicionar campos ao domínio sempre que surge uma necessidade nova.

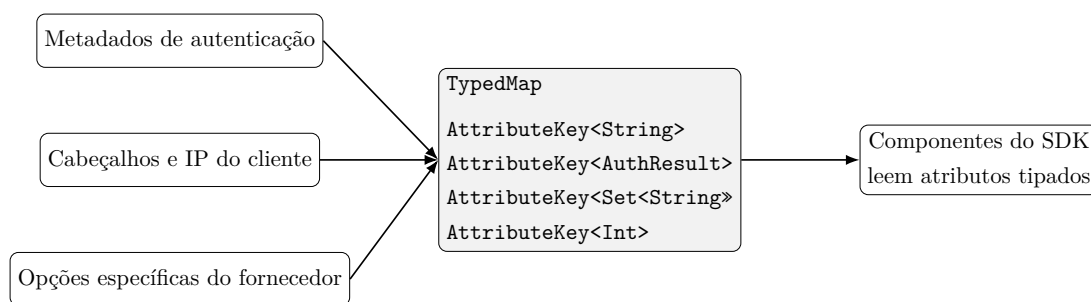


Figura 3.4: Utilização da `TypedMap` para transportar atributos extensíveis

No domínio de inferência, a `TypedMap` aparece em `providerOptions`, permitindo preservar dados que não pertencem ao subconjunto comum mas não devem ser perdidos durante a tradução. Fora da inferência estrita, a mesma estrutura permite transportar metadados entre componentes sem criar dependências diretas entre eles. Esta característica é útil para a evolução futura do SDK, porque permite que mecanismos transversais, como autorização, observabilidade

ou políticas de execução, usem atributos adicionais sem forçar alterações constantes nas classes centrais do domínio.

Capítulo 4

Arquitetura e Desenho do Sistema

4.1 Arquitetura Top-Level

A arquitetura top-level distingue claramente o produto principal desenvolvido, o **OmniAI SDK**, da aplicação de validação, a **OmniAI Gateway**. O SDK contém o domínio comum, os contratos, os adapters, o *dispatcher*, a *pipeline*, os *interceptors*, a DSL de montagem e as abstrações de transporte. A OmniAI Gateway é um POC construído sobre esse SDK: carrega configuração, cria o grafo de componentes, instala os conectores Ktor e expõe endpoints HTTP compatíveis com os contratos estudados.

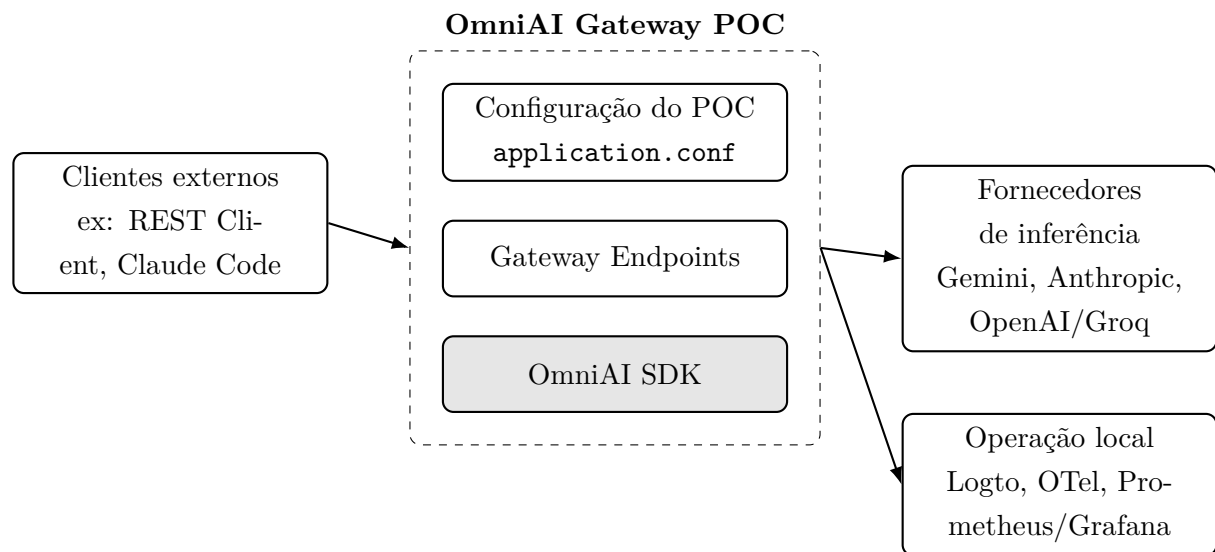


Figura 4.1: Arquitetura top-level: POC OmniAI Gateway a instanciar e demonstrar o OmniAI SDK

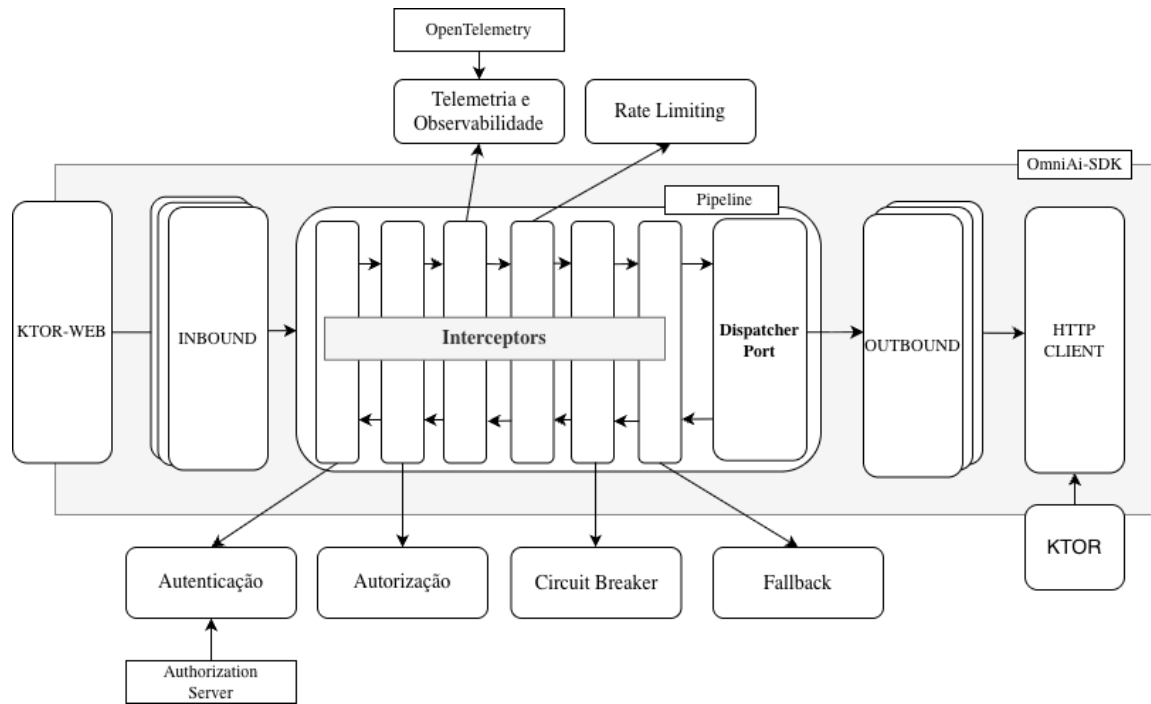


Figura 4.2: Visão da integração entre Ktor, inbounds, pipeline, interceptors, outbounds e HTTP client no OmniAI SDK

A Figura 4.1 mostra que o POC não contém a lógica essencial de tradução, despacho ou resiliência. Essa lógica pertence ao SDK e é apenas instanciada pela aplicação final. A Figura 4.2 aproxima a visão interna: um conector HTTP recebe o corpo do pedido, converte-o para um DTO de contrato, invoca um *inbound adapter*, e este chama um **DispatcherPort**. A partir desse ponto, a execução pode seguir por um dispatcher direto ou por um dispatcher apoiado numa *pipeline*. No final, um *outbound adapter* traduz o pedido comum para o contrato do fornecedor e utiliza um cliente HTTP abstrato para executar a chamada real.

Esta organização permite que aplicações já compatíveis com APIs existentes possam ser integradas com esforço reduzido. Por exemplo, uma aplicação preparada para a API Anthropic pode alterar o seu `baseUrl` para apontar para a OmniAI Gateway, mantendo o contrato esperado pelo cliente. O mesmo princípio aplica-se a clientes compatíveis com OpenAI e a cenários Gemini suportados pelo POC. A arquitetura foi também pensada para escalamento horizontal: o SDK evita estado global obrigatório, concentra estado transitório no contexto do pedido e define portas para substituir stores em memória por backends partilhados quando a Gateway é executada em múltiplas instâncias.

4.2 Arquitetura Hexagonal (Ports & Adapters)

A Arquitetura Hexagonal foi escolhida para separar a lógica de domínio dos detalhes de transporte, dos contratos externos e das bibliotecas concretas. A formulação original de Cockburn descreve a aplicação no centro e os mecanismos externos ligados através de portas e adapters [21]. Esta abordagem é particularmente adequada a uma Gateway de IA, porque o sistema é dominado por integrações: clientes externos, APIs de fornecedores, servidores de autorização,

telemetria e mecanismos de transporte.

Uma alternativa seria uma arquitetura em camadas tradicional, com controladores, serviços e repositórios. Essa opção seria simples no início, mas tenderia a concentrar traduções de contratos, chamadas HTTP e regras operacionais no mesmo serviço de aplicação. Outra alternativa seria decompor cedo a solução em vários serviços independentes. Essa abordagem poderia escalar equipes e deploys, mas introduziria complexidade operacional excessiva para um SDK cujo objetivo principal é ser reutilizável dentro de diferentes aplicações. A opção hexagonal oferece um compromisso mais adequado: mantém uma estrutura modular e testável sem obrigar a transformar cada responsabilidade num serviço remoto. No contexto de gateways, esta separação também acompanha o padrão de API Gateway como camada de integração e política transversal [11, 12, 13].

4.2.1 Contracts

Os módulos `contracts:openai`, `contracts:anthropic` e `contracts:gemin`i modelam os formatos de pedido, resposta e eventos de cada fornecedor. Estes contratos são DTOs no sentido em que representam dados transportados na fronteira do sistema, mas não são apenas DTOs genéricos. São chamados *contracts* porque codificam uma compatibilidade externa concreta: nomes de campos, estruturas JSON, eventos de *streaming*, campos opcionais, formatos de erro e convenções específicas de cada API.

Esta distinção é importante. Um DTO interno poderia ser livremente alterado sem impacto externo; um contrato compatível com OpenAI, Anthropic ou Gemini precisa de respeitar o formato que clientes e fornecedores esperam. Ao isolar estes contratos em módulos próprios, o SDK evita que detalhes como `tool_calls`, `content_block_delta` ou `functionDeclarations` se espalhem pelo domínio comum.

4.2.2 Inbounds

Os *inbound adapters* recebem pedidos no formato do cliente e traduzem-nos para `CommonRequest`. Cada inbound implementa a porta `InboundPort` e recebe um `DispatcherPort`, não uma implementação concreta de pipeline ou de fornecedor. Além disso, cada inbound usa uma implementação de `InboundTranslator`, responsável por converter o objeto de entrada para domínio e por converter a resposta comum de volta para o contrato do cliente.

Esta decisão evita que o inbound exponha diretamente uma Web API. O SDK fornece adapters que trabalham sobre objetos de transferência, e a exposição HTTP fica noutra adapter, como o `gateway-ktor-server`. Assim, um servidor Ktor, uma função serverless, um handler Express ou outro mecanismo de entrada podem chamar o mesmo inbound desde que consigam construir o DTO esperado. Esta separação reduz a dependência do domínio e dos inbounds face a bibliotecas HTTP externas.

4.2.3 Outbounds

Os *outbound adapters* fazem o caminho inverso: recebem um `CommonRequest`, traduzem-no para o contrato do fornecedor real e executam a chamada externa. Cada outbound implementa

`OutboundPort`, identifica `provider`, `model` e `key`, e delega a tradução num `OutboundTranslator`. A implementação atual inclui outbounds para OpenAI, Anthropic e Gemini.

Ao contrário dos inbounds, os outbounds precisam de comunicar com serviços externos. Por isso recebem uma implementação de `HttpTransportClient`. A implementação fornecida pelo SDK é baseada em Ktor, mas a porta permite substituir o transporte por outro cliente HTTP, por um mock em testes, por um cliente instrumentado ou por uma implementação específica de plataforma. Esta inversão de dependência é o que permite testar adapters sem depender de chamadas reais e evoluir o runtime sem reescrever o domínio.

4.2.4 Dispatcher

O `DispatcherPort` é a porta chamada pelos inbounds para executar um `CommonRequest`. A sua responsabilidade não é, por si só, conter inteligência de roteamento; é despachar o pedido para uma estratégia de execução. No SDK é fornecida uma implementação simples, criada por `routingDispatcherFactory`, que procura o outbound cujo `provider` e `model` correspondem aos campos do pedido e envia a chamada para esse adapter.

Como se trata de uma interface, a arquitetura permite outras implementações com objetivos diferentes. Um consumidor do SDK pode fornecer um dispatcher direto, um dispatcher com seleção por custo, um dispatcher que consulta políticas externas ou um dispatcher que aplica regras mais complexas. A *pipeline* fornecida pelo SDK é ela própria encaixada através de um adapter, `PipelineBackedDispatcher`, preservando a ideia de que os inbounds dependem apenas de `DispatcherPort`.

4.3 Ecossistema do SDK

O SDK está organizado como um projeto Gradle multi-módulo. Os módulos principais incluem `core`, `contracts`, `inbound`, `outbound`, `interceptors`, `mcp-broker`, `dispatcher`, `gateway-client`, `gateway-ktor-server` e `contracts:ktor-http`. Esta divisão permite evoluir cada área com responsabilidades claras: domínio e portas em `core`, contratos externos em `contracts`, adapters de entrada e saída em módulos próprios, preocupações transversais em `interceptors`, montagem declarativa em `gateway-client` e exposição HTTP/Ktor em `gateway-ktor-server`.

O módulo `contracts:ktor-http` fornece a abstração de transporte `HttpTransportClient` e a implementação `KtorHttpTransportClient`. Esta camada encapsula chamadas HTTP, SSE, serialização, metadados de resposta, erros de rede e erros de parsing. Ao colocá-la atrás de uma interface, os outbounds não ficam presos a uma implementação específica de cliente HTTP.

O módulo `gateway-client` disponibiliza uma DSL para montar uma Gateway. A aplicação consumidora pode declarar outbounds, interceptors, autenticação, autorização e modo de execução. A DSL permite escolher entre `useNativePipeline`, que monta a pipeline e o dispatcher padrão, e `useCustomDispatcher`, que permite substituir totalmente a estratégia de execução. Esta flexibilidade é essencial para um SDK: o POC usa a montagem nativa, mas outro consumidor pode querer um dispatcher próprio.

A aplicação `OmniAiGateway` foi desenvolvida como POC de validação do SDK. O uso de *composite build* do Gradle, através de `includeBuild("../OmniAi-SDK")` [22], não pretende

simular a forma final de consumo em produção. A sua função foi acelerar o desenvolvimento do projeto final, permitindo compilar SDK e POC em conjunto, testar alterações imediatamente e evitar o ciclo pesado de publicar uma versão do SDK, atualizar dependências do POC e voltar a compilar. Numa distribuição real, a expectativa é que o SDK seja consumido como dependência versionada, nomeadamente via Maven para JVM e via NPM para Node.js/TypeScript.

Capítulo 5

Implementação e Evolução Técnica

5.1 Stack Tecnológica e Metodologias

O SDK foi desenvolvido em Kotlin Multiplatform com targets JVM e JavaScript/Node.js. A escolha de KMP permite partilhar o domínio, os contratos, as portas, os adapters principais e parte da lógica transversal entre plataformas, mantendo implementações específicas quando o ambiente o exige [16]. A serialização dos contratos é baseada em Kotlinx Serialization, o que facilita a definição explícita dos modelos JSON estudados.

O Ktor é usado em dois pontos diferentes, que importa distinguir. Primeiro, o SDK fornece uma implementação HTTP baseada em Ktor, através de `KtorHttpClient`, usada pelos outbounds para executar pedidos HTTP e SSE. Segundo, o SDK fornece o módulo `gateway-ktor-server`, que disponibiliza conectores Ktor para expor rotas compatíveis com OpenAI, Anthropic e Gemini. Logo, importa clarificar que o Ktor não constitui o SDK, representando apenas uma das suas implementações concretas, a qual é adotada de forma nativa pelo POC OmniAI Gateway.

A OmniAI Gateway, por sua vez, é uma aplicação Ktor separada, construída como prova de conceito para demonstrar o uso do OmniAI SDK num sistema real.

As ferramentas de desenvolvimento incluíram Git, IntelliJ IDEA, o cliente HTTP do IDE para pedidos REST, Docker Compose para Logto, PostgreSQL, OpenTelemetry Collector, Prometheus e Grafana, além de clientes reais como Claude Code e SDKs oficiais. Também foram usados assistentes de apoio, como Gemini no browser e agentes de programação, para pesquisa, comparação de documentação e aceleração de tarefas, mantendo a validação no código, nos testes e nos cenários executados. A pipeline de CI existente executa `./gradlew clean build`; como evolução natural, prevê-se uma pipeline de CD para publicar versões do SDK em Maven e NPM.

5.2 A Primeira Iteração da Gateway

A primeira iteração do projeto consistiu numa Gateway HTTP construída diretamente, responsável por receber pedidos de clientes e encaminhá-los para fornecedores de inferência. Esta fase foi útil para validar rapidamente a viabilidade da solução, testar chamadas reais e compreender diferenças práticas entre OpenAI, Anthropic e Gemini.

À medida que o sistema cresceu, essa abordagem revelou acoplamento excessivo. A tradução entre contratos, a configuração dos fornecedores, a autenticação, o tratamento de erros e o encaminhamento de pedidos começaram a concentrar-se na própria aplicação. Cada novo fornecedor ou preocupação transversal exigia alterações no mesmo conjunto de componentes, tornando o código difícil de evoluir e testar.

A reorganização separou o produto principal, o **OmniAI SDK**, da aplicação demonstradora, a **OmniAI Gateway**. O OmniAI SDK passou a concentrar o domínio comum, contratos, adapters, portas, *dispatcher*, *pipeline*, *interceptors*, DSL e abstrações HTTP. A OmniAI Gateway passou a ser um POC executável, usado para demonstrar o consumo do SDK num sistema real: carrega configuração, instancia os componentes necessários, instala os conectores Ktor e expõe endpoints HTTP. Esta aplicação não reimplementa a lógica central da biblioteca; valida que essa lógica pode ser composta, configurada e executada fora dos módulos internos do SDK.

Esta separação também influenciou o desenho para escalamento horizontal. O SDK foi projetado para manter o estado do pedido dentro de estruturas transitórias, como `GatewayContext` e `TypedMap`, e para expor portas onde existe estado partilhável, como caches, rate limiting, circuit breaker, autenticação ou telemetria. Consequentemente, a arquitetura permite que uma implementação inicial utilize armazenamento em memória para simplificar o POC; por outro lado, num cenário de escalamento horizontal, esses mecanismos podem ser substituídos por backends partilhados, sem implicar qualquer alteração nos inbounds, outbounds ou contratos.

5.3 Decisões de Desenho e Alternativas Descartadas

A análise de *trade-offs* partiu de três critérios: proteger a estabilidade da versão demonstrável, manter o SDK reutilizável fora do POC e evitar que funcionalidades experimentais contaminassem o domínio comum. As decisões seguintes não representam funcionalidades abandonadas, mas sim escolhas de âmbito para manter a primeira versão estável, testável e alinhada com o domínio principal do projeto:

- **Geração de Bridges com KSP (Kotlin Symbol Processing):** Foi ponderada a utilização de KSP para gerar automaticamente *bridges* entre Kotlin e TypeScript/Node.js. A alternativa foi descartada porque a configuração, a compatibilidade multiplataforma e os casos limite de tipagem consumiriam tempo desproporcional. Optou-se por uma fachada `@JsExport` controlada, suficiente para preparar o target JavaScript sem comprometer o prazo.
- **Injeção Automática de Ferramentas via SSE:** Também foi avaliada a possibilidade de a Gateway intercetar *tool calls* durante *streams*, executar ferramentas e injetar resultados automaticamente no fluxo. Esta opção exigiria gerir estado de conversação, suspender e retomar *streams*, lidar com concorrência e manter compatibilidade com fornecedores que emitem eventos diferentes. A capacidade ficou identificada como trabalho futuro, mantendo a primeira versão focada em contratos, adapters, segurança, observabilidade e resiliência.

5.4 Evolução Arquitetural: De *Inference Service* a *Dispatcher*

A primeira iteração concentrava comunicação, tradução de contratos e roteamento num componente designado por `InferenceService`. Embora esta abordagem tenha sido eficaz para prototipagem, rapidamente se aproximou de um *God Object*: cada fornecedor, métrica, regra de segurança ou política de erro aumentava a responsabilidade do mesmo serviço.

O problema principal não era apenas o tamanho do componente, mas a direção das dependências. Se o serviço conhecesse diretamente contratos HTTP, clientes externos, regras de seleção de fornecedor e políticas transversais, qualquer alteração numa dessas áreas teria impacto no centro do sistema. Isto dificultaria testes unitários, tornaria mais cara a adição de novos adapters e reduziria a utilidade do SDK como biblioteca reutilizável.

A refatorização substituiu esse serviço por um contrato pequeno, o `DispatcherPort`. A responsabilidade do dispatcher é receber um `CommonRequest` e despachá-lo para uma estratégia de execução. A implementação padrão fornecida, criada por `routingDispatcherFactory`, faz um despacho simples: procura um outbound com o mesmo fornecedor e modelo presentes no pedido. Por se tratar de uma interface, o SDK não força esta estratégia; um consumidor pode substituir o dispatcher por outra implementação sem alterar inbounds ou contratos.

Esta mudança reduziu o acoplamento e clarificou a arquitetura. Os inbounds passaram a conhecer apenas a porta `DispatcherPort`; os outbounds passaram a concentrar tradução e comunicação externa; e as preocupações transversais foram deslocadas para uma estrutura própria, a *pipeline* de *interceptors*. O dispatcher passou, portanto, a ser o ponto de decisão da execução, enquanto a pipeline passou a ser apenas uma forma opcional de decorar essa execução com responsabilidades operacionais.

5.5 Evolução para a Pipeline de Interceptors

A *pipeline* foi implementada como um adapter de dispatcher. Isto é, os inbounds não recebem uma pipeline; recebem sempre um `DispatcherPort`. Quando a aplicação escolhe `useNativePipeline`, o SDK monta uma `GatewayPipeline` com interceptors e um dispatcher terminal, e depois expõe essa cadeia através de `PipelineBackedDispatcher`. Se a aplicação não quiser a pipeline, pode fornecer diretamente um dispatcher próprio através de `useCustomDispatcher`.

Esta decisão evita que a pipeline se torne uma dependência obrigatória da arquitetura. Um consumidor que pretenda apenas tradução de contratos e despacho direto pode usar um dispatcher simples. Um consumidor que precise de segurança, observabilidade e resiliência pode ativar a pipeline nativa. Um consumidor com necessidades próprias pode implementar outro `DispatcherPort`, por exemplo com seleção por custo, políticas externas, filas internas ou integração com outro orquestrador.

Modo de montagem	Componente visto pelos inbounds	Justificação
Dispatcher direto	Uma implementação de <code>DispatcherPort</code> .	Útil quando se pretende uma Gateway mínima, com tradução e envio para outbounds sem lógica transversal.
Pipeline nativa	<code>PipelineBackedDispatcher</code> , também exposto como <code>DispatcherPort</code> .	Permite aplicar autenticação, autorização, métricas, rate limiting, circuit breaker, fallback e logging sem alterar adapters.
Dispatcher customizado	Qualquer implementação externa de <code>DispatcherPort</code> .	Mantém o SDK aberto a estratégias futuras, incluindo roteamento por custo, latência, plano de cliente ou políticas de negócio.

Tabela 5.1: Formas de expor a execução aos inbounds sem os acoplar à pipeline

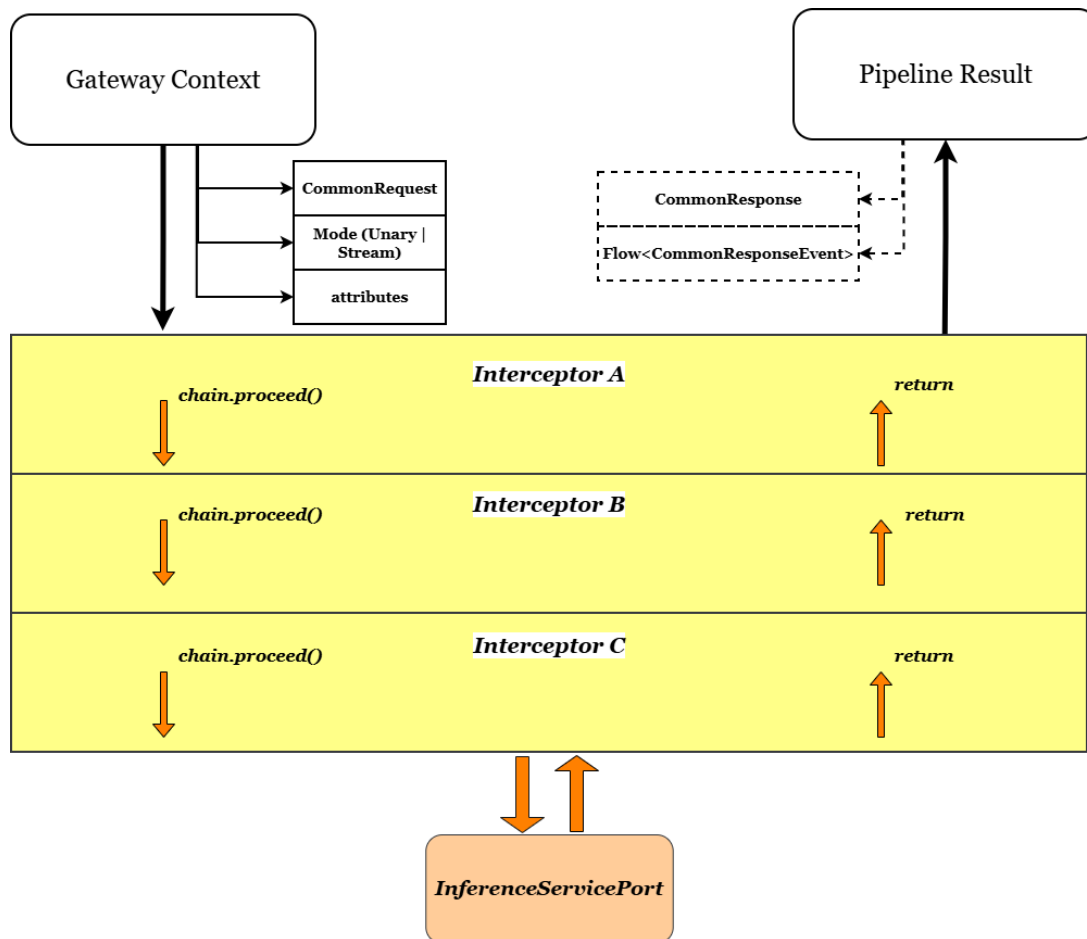


Figura 5.1: Visão conceitual da pipeline de interceptors

Esta forma de encaixe mantém a arquitetura simples. Para um caso mínimo, a Gateway pode operar com um dispatcher direto. Para um caso operacional, a pipeline permite adicionar autenticação, autorização, métricas, limites, *circuit breaker*, *fallback*, logging e tracing sem alterar adapters. O modelo é semelhante ao dos filtros de Servlets [18]: cada filtro recebe um contexto, decide se continua a cadeia e pode executar lógica antes e depois do próximo elemento.

A diferença é que, no OmniAI SDK, o contexto não representa apenas um pedido HTTP; representa uma chamada de inferência já normalizada, incluindo fornecedor, modelo, modo de execução, atributos tipados e eventual fluxo de *streaming*.

A *pipeline* é composta por `GatewayPipeline`, `GatewayPipelineChain`, `Interceptor` e `GatewayContext`. O `GatewayContext` transporta o pedido comum, atributos tipados e modo de execução, podendo representar chamadas síncronas ou *streaming*. A cadeia executa os *interceptors* por ordem e, no fim, invoca o dispatcher terminal.

Esta abordagem também prepara o escalamento horizontal. A pipeline não exige estado global obrigatório; cada pedido transporta o seu contexto. Onde existe estado partilhável, como rate limiting ou circuit breaker, o SDK define stores substituíveis. A implementação em memória é adequada para desenvolvimento e instância única, mas pode ser trocada por Redis, base de dados ou outro backend coordenado em implantações distribuídas. Assim, várias instâncias da OmniAI Gateway podem partilhar contadores, estados de circuito e metadados operacionais sem alterar o contrato dos inbounds nem a forma como os clientes chamam a Gateway.

5.6 Arquitetura Interna dos Interceptors

O módulo `interceptors` inclui implementações pré-construídas para preocupações operacionais e de segurança. Todos seguem o mesmo contrato: recebem um `GatewayContext`, consultam ou acrescentam atributos à `TypedMap`, decidem se chamam `chain.proceed(context)` e podem observar ou transformar o `PipelineResult`. Esta estrutura permite colocar lógica transversal antes e depois da inferência sem acoplar essa lógica aos adapters de entrada, aos adapters de saída ou ao dispatcher.

A ordem de execução é configurável. No target JVM, o SDK suporta a anotação `InterceptorPriority`, usada para executar primeiro interceptors com prioridade superior. No target JavaScript, onde a reflexão sobre anotações é limitada, a ordem segue a sequência definida pela DSL. Assim, autenticação e autorização podem ocorrer no início da cadeia, enquanto métricas, tracing e logging podem envolver a execução completa.

A motivação para estes interceptors não foi apenas organizar código, mas tornar explícitas políticas que, numa Gateway de IA, afetam segurança, custo e disponibilidade. A Tabela 5.2 resume essa relação.

Interceptor	Problema tratado	Motivo para estar na Gateway
Telemetria	Falta de visibilidade sobre latência, erros, tokens e fornecedores.	A Gateway observa todas as chamadas e consegue medir comportamento transversal sem alterar clientes ou adapters.
Autenticação	Necessidade de validar quem apresenta o token antes de consumir modelos pagos.	A Gateway é o recurso protegido e deve validar tokens antes de encaminhar tráfego para fornecedores.
Autorização	Um token válido não implica permissão para todos os modelos ou operações.	A decisão por cliente, ação, fornecedor e modelo deve ser centralizada antes da inferência.
Rate limiting	Risco de abuso, picos de carga e custos inesperados.	A Gateway é o ponto natural para aplicar quotas por cliente, plano, fornecedor ou modelo.
Circuit breaker	Dependências externas podem ficar lentas ou indisponíveis.	A Gateway evita insistir em fornecedores em falha e protege o restante sistema.
Fallback	Uma falha de fornecedor não deve, quando existe alternativa, falhar imediatamente a experiência do cliente.	A Gateway conhece os outbounds disponíveis e pode tentar alternativas mantendo o contrato do cliente.

Tabela 5.2: Justificação dos interceptors fornecidos pelo SDK

5.6.1 Interceptor de Telemetria e Observabilidade

O `MetricsInterceptor` recolhe dados operacionais essenciais num sistema que intermedeia chamadas a LLMs: latência, sucesso, erro, fornecedor, modelo, cliente e consumo de tokens. Estes dados são importantes porque uma Gateway centraliza tráfego de vários clientes e fornecedores; sem telemetria, não é possível perceber custos, degradação, falhas por modelo ou impacto de políticas como *fallback*.

A implementação distingue chamadas unárias de *streaming*. Em respostas progressivas, o interceptor não bloqueia a execução; usa operadores de fluxo para observar eventos e emitir métricas no encerramento do *stream*. Assim mede a duração total da interação, e não apenas o tempo até ao primeiro fragmento.

A arquitetura segue inversão de dependência através da porta `MetricsPort`. O OmniAI SDK define instrumentos comuns, como `counter`, `histogram` e `upDownCounter`, enquanto o adapter concreto no target JVM exporta para OpenTelemetry. O uso do OpenTelemetry Protocol (OTLP) foi escolhido porque o OTLP define a codificação, transporte e entrega de dados de telemetria entre fontes, coletores e backends [23]. Esta escolha evita prender o OmniAI SDK a Prometheus, Grafana, Datadog ou outro backend específico: o SDK emite telemetria num formato aberto, o Collector recebe e processa esses dados, e a infraestrutura decide para onde exportar.

Esta decisão é especialmente importante numa Gateway de IA. O mesmo OmniAI SDK pode ser usado por uma aplicação simples em desenvolvimento, por um POC local com Prometheus/Grafana ou por uma instalação futura com outro backend de observabilidade. A Open-

Telemetry apresenta-se precisamente como especificação aberta e agnóstica de fornecedor [24]; por isso, a instrumentação fica no OmniAI SDK, mas a escolha operacional do backend permanece fora do domínio.

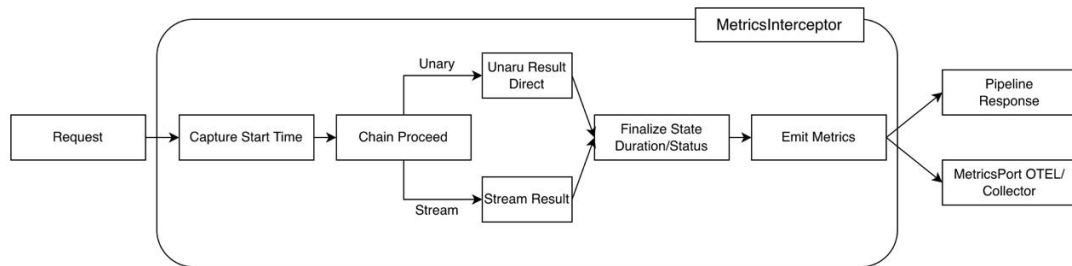


Figura 5.2: Fluxo interno de processamento e lógica de observabilidade para pedidos unários e streaming

Para adicionar uma métrica, a aplicação usa a DSL do `gateway-client`. O programador pode ativar a latência padrão, acrescentar atributos globais e definir métricas próprias com uma função `value`, que extrai o valor a partir do contexto e do resultado, e uma função `tags`, que acrescenta dimensões de análise.

```

metrics {
  metricsPort = telemetryRuntime.metricsPort
  tracer = telemetryRuntime.tracer

  attributes {
    include(ClientIpMetadataKey, alias = "client.ip")
    attribute("sdk.version") { _, _ -> "1.0.0" }
  }

  defaultLatency {
    enabled = true
    name = "gateway.inference.request.duration"
  }

  customMetrics {
    counter(
      name = "gateway.llm.tokens",
      description = "Total de tokens consumidos",
      unit = "{tokens}"
    ) {
      value { _, result ->
        (result as? PipelineResult.Unary)?.response?.usage?.totalTokens
      }
    }
  }
}

```

}

Este desenho separa três responsabilidades: a definição da métrica na DSL, a recolha no interceptor e a exportação no adapter de telemetria. Também facilita testes, porque `NoOpMetricsPort` pode ser usado quando a observabilidade está desligada.

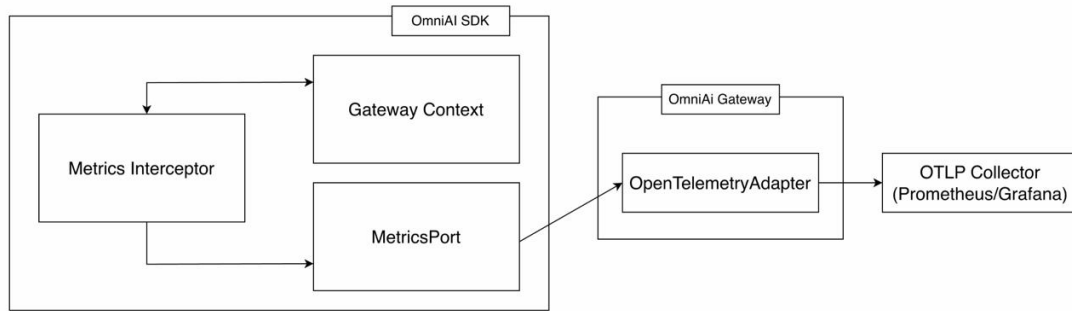


Figura 5.3: Observabilidade e métricas operacionais da OmniAI Gateway

5.6.2 Interceptor de Autenticação

A autenticação é realizada pelo `AuthenticationInterceptor`. A Gateway atua como *Resource Server*: recebe pedidos para recursos protegidos, extrai o *access token* do cabeçalho `Authorization: Bearer <token>` e valida se esse token foi emitido por um servidor de autorização confiável [25, 26, 27]. Esta responsabilidade é essencial numa AI Gateway, porque a camada central passa a proteger acesso a modelos, custos, quotas e dados operacionais.

A Gateway não deve atuar como *Authorization Server*: não emite tokens, não gere credenciais dos clientes e não substitui Logto ou Keycloak. O seu papel é o de API protegida. Por isso, valida tokens recebidos, confirma se foram emitidos para a audiência esperada e só depois permite que o pedido avance na pipeline para os próximos interceptors. Esta separação reduz responsabilidade de segurança dentro do SDK e permite integrar diferentes fornecedores de identidade sem alterar o contrato dos clientes.

O fluxo inicia-se no adapter HTTP, que propaga o token para o domínio através de metadados. O interceptor classifica o token: se tiver três segmentos, é tratado como JWT; caso contrário, como token opaco. Esta distinção permite escolher entre validação local por assinatura e validação remota por introspeção.

No modo *OIDC Discovery* (RFC 8414), a Gateway obtém dinamicamente metadados do *Authorization Server*, como *issuer*, *jwks_uri* e endpoint de introspeção [28]. Todo este fluxo encontra-se ilustrado na Figura 5.4. Para JWTs, valida o *kid*, obtém a chave pública no JWKS, verifica assinatura e confirma *claims* como *exp*, *nbf*, *iss* e *aud* [29, 30, 31]. Para tokens opacos, usa introspeção OAuth2 [32].

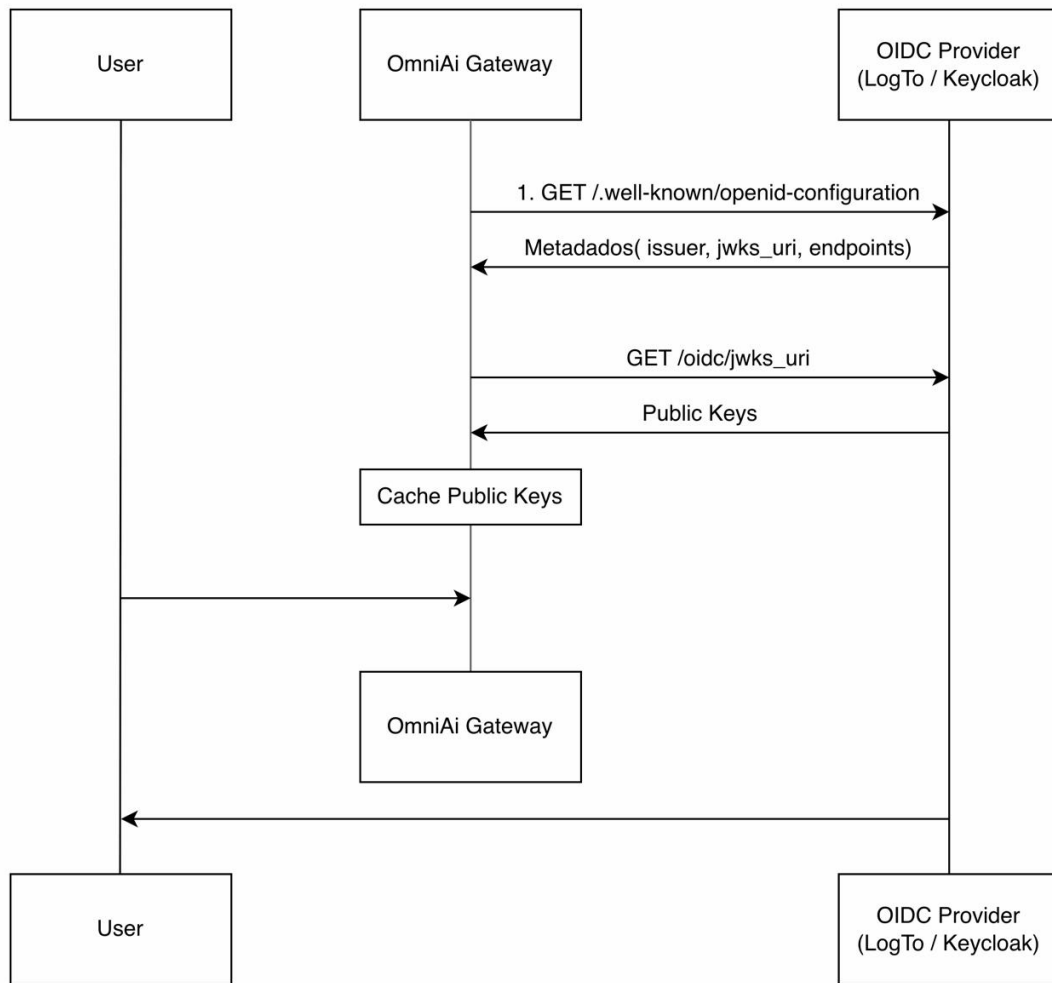


Figura 5.4: Sequência OIDC: descoberta, JWKS e cache de chaves

Para reduzir latência e chamadas repetidas ao servidor de autorização, a implementação inclui `InMemoryKeyCache`, rotação de chaves em *background* e *cooldown* para evitar consultas excessivas quando uma chave não é encontrada. No caso de tokens opacos, o `IntrospectionCache` usa *positive caching* para tokens ativos e *negative caching* para tokens inválidos. Os tokens opacos são guardados em cache através de *hash*, evitando manter credenciais em texto limpo em memória.

O resultado é encapsulado em `AuthValidationResult` e guardado na `TypedMap` sob a chave `AUTH_RESULT_KEY`. Isto permite que autorização, métricas e logging reutilizem a identidade já validada.

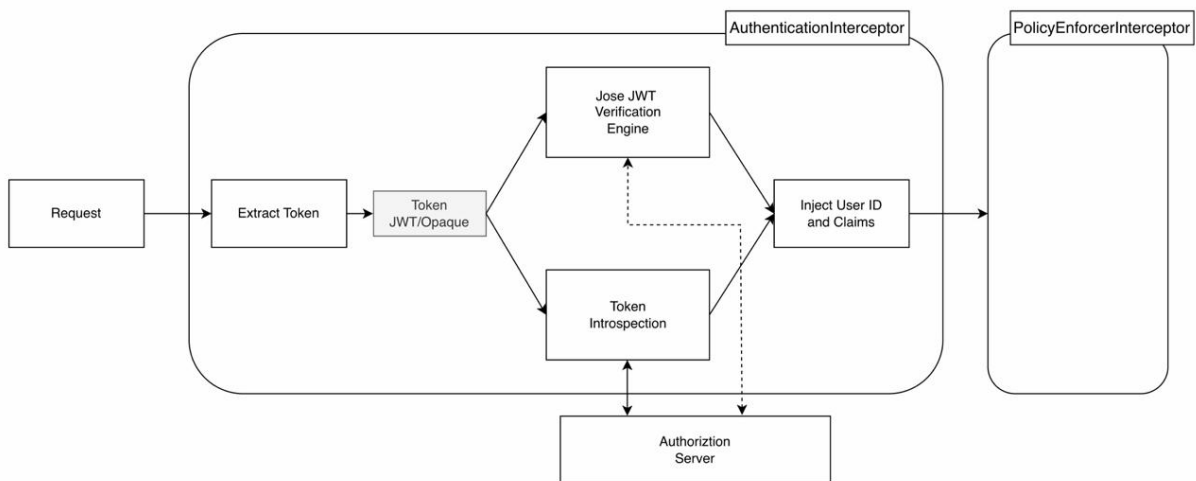


Figura 5.5: Fluxo interno do interceptor de autenticação

A Gateway final usa Logto como Authorization Server local [33], executado em Docker juntamente com PostgreSQL, OpenTelemetry Collector, Prometheus e Grafana. Foram também realizados testes exploratórios com Keycloak [34]. A escolha final do Logto no POC deveu-se à simplicidade de configuração local, suporte a aplicações Machine-to-Machine e facilidade de gerir API Resources durante a validação.

5.6.3 Interceptor de Autorização

A autorização é separada da autenticação porque responder a “quem é o utilizador?” não é o mesmo que responder a “o que este utilizador pode fazer?”. A autenticação valida identidade; a autorização decide permissões sobre uma ação e um recurso. Esta separação é importante numa Gateway multi-cliente, onde um token válido pode não ter permissão para usar todos os modelos, fornecedores ou operações. A distinção entre autenticação e autorização é também a base dos modelos modernos de identidade: a primeira prova identidade, enquanto a segunda concede permissão sobre dados ou ações protegidas [35].

No contexto do projeto, esta separação evita dois problemas. Primeiro, impede que a validação de token seja confundida com autorização total: um cliente autenticado pode estar limitado a um subconjunto de modelos, fornecedores ou quotas. Segundo, permite trocar a origem das decisões de autorização sem alterar a autenticação. Embora numa fase inicial a decisão possa operar de forma local e simples, o sistema está desenhado para que, numa iteração futura, essa mesma decisão possa ser assegurada por um PDP externo, tal como o AuthZen ou o OPA.

No OmniAI SDK, a autorização é centralizada no `PolicyEnforcerInterceptor`. O componente atua como *Policy Enforcement Point* (PEP): interceta o pedido, constrói um `AuthorizationInput` e delega a decisão a um *Policy Decision Point* (PDP) através da porta `PolicyDecisionPointPort`. Podemos ver todo este fluxo na figura 5.6.

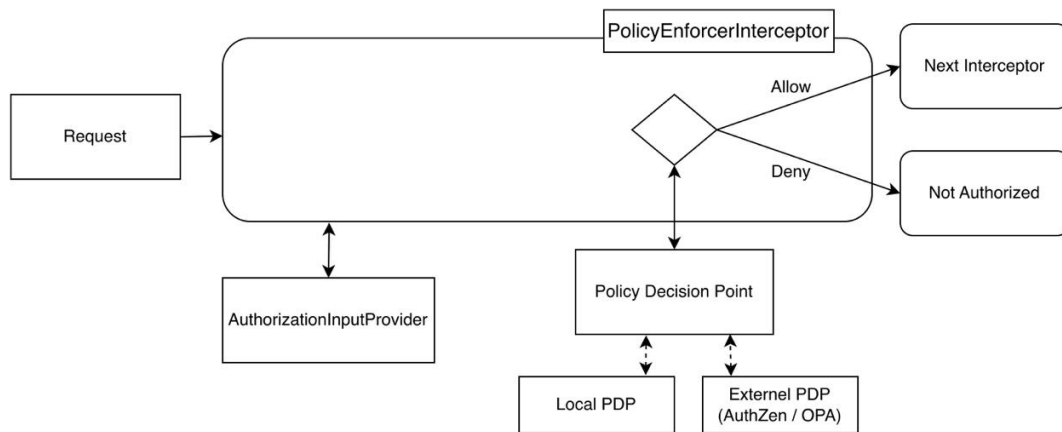


Figura 5.6: Arquitetura do interceptor de autorização: separação entre PEP e PDP

O **AuthorizationInput** segue o padrão **Subject-Action-Resource (SAR)**. O **Subject** representa o utilizador ou aplicação, incluindo identificador, roles e atributos vindos do token. A **Action** representa a operação, como `chat_completion`. O **Resource** identifica o alvo, como fornecedor ou modelo. O **Context** acrescenta atributos ambientais, como IP, tempo ou metadados do pedido. A porta do PDP permite integrar motores diferentes, incluindo políticas internas, AuthZen, OPA ou outro serviço externo. Se a decisão for *Deny*, a pipeline é interrompida antes de qualquer chamada ao fornecedor [36].

5.6.4 Interceptor de Rate Limiting

O **RateLimitInterceptor** aplica limites de utilização antes da chamada ao fornecedor. Numa AI Gateway, este mecanismo tem dois objetivos principais: proteger o sistema contra abuso ou tráfego excessivo e controlar custos associados a chamadas de inferência. A limitação de taxa é uma estratégia comum para impor um teto de ações repetidas num intervalo temporal, reduzindo abuso de APIs, força bruta e carga excessiva [37].

A importância deste interceptor é maior do que numa API tradicional, porque cada pedido pode representar custo financeiro direto, consumo de quotas externas e ocupação prolongada de ligações em *streaming*. Sem rate limiting, um cliente mal configurado, uma automação em ciclo ou um utilizador abusivo pode saturar a Gateway, esgotar quotas nos fornecedores e degradar o serviço para clientes legítimos. O interceptor permite aplicar justiça operacional: cada cliente, plano, modelo ou fornecedor pode ter limites próprios.

A lógica é orientada por políticas. Cada **RateLimitPolicy** extrai dados do **GatewayContext** e devolve **RateLimitTarget**. Um target contém uma chave, como cliente/modelo/plano, e uma **RateLimitDefinition**, composta por limite, tipo e janela temporal. Isto permite políticas diferentes por cliente, plano, modelo ou fornecedor.

O store atual em memória usa buckets protegidos por **Mutex**. Quando um pedido chega, o interceptor tenta consumir uma unidade em todos os buckets aplicáveis. Se todos permitirem consumo, a cadeia continua. Se algum limite estiver esgotado, devolve **TooManyRequestsError** com **retryAfter**. Esta resposta permite que clientes bem comportados reduzam ritmo em vez de insistirem. Para desenvolvimento e instância única, este modelo é suficiente; em escalamento

horizontal, o contrato `RateLimitStore` permite substituir a implementação por Redis ou outro backend compartilhado. Essa substituição é necessária quando várias instâncias da Gateway devem partilhar a mesma visão de quota.

5.6.5 Interceptor de Circuit Breaker

O `CircuitBreakerInterceptor` protege a Gateway contra chamadas repetidas a um outbound em falha. O padrão de *circuit breaker* é usado para lidar com falhas em serviços remotos, evitando insistir numa dependência que está indisponível e dando tempo para recuperação [38, 39, 40]. Numa Gateway de IA, esta proteção é especialmente útil porque fornecedores externos podem falhar por indisponibilidade, timeouts, quotas ou erros 5xx.

A alternativa simples seria repetir pedidos até obter sucesso. Essa abordagem é perigosa quando o fornecedor está realmente indisponível: as tentativas acumulam latência, ocupam threads ou corrotinas, aumentam custos e podem agravar a falha no serviço externo. O circuit breaker foi criado para evitar esse comportamento. Quando a falha ultrapassa um limiar, o circuito abre e a Gateway passa a falhar rapidamente aquele destino, libertando recursos e permitindo que mecanismos como fallback escolham outro outbound.

Para cada combinação de fornecedor/modelo, o interceptor identifica o `outbound.key` e consulta o `CircuitBreakerStore`. Se o circuito estiver `OPEN`, o pedido falha rapidamente com `ApiDownError` e o outbound é acrescentado ao conjunto de outbounds negados no contexto. Esta falha rápida reduz latência, evita consumo de recursos e dá ao *fallback* informação para não tentar novamente o mesmo destino.

Quando a chamada prossegue, o interceptor observa o resultado. Erros incrementam o contador de falhas; ao atingir `failureThreshold`, o circuito passa para `OPEN`. Após um período de recuperação, o estado `HALF_OPEN` permite testar um número limitado de pedidos antes de voltar ao estado `CLOSED`, evitando inundar um fornecedor que ainda está a recuperar [38]. A configuração inclui `failureThreshold`, `resetTimeoutMs` e `halfOpenTestRequests`. A implementação atual cobre a abertura e fecho básico do circuito; a evolução temporal mais completa do estado `HALF_OPEN` é uma melhoria natural.

5.6.6 Interceptor de Fallback

O `FallbackInterceptor` permite tentar alternativas quando a chamada principal falha. Em sistemas de IA, *fallback* ao nível de modelo ou fornecedor é relevante porque falhas nas APIs de inferência fornecidas pelas LLM's são um cenário frequente em produção: rate limits, indisponibilidade, quotas, timeouts ou modelos descontinuados podem impedir a resposta do fornecedor principal [41, 42]. Um gateway é o ponto certo para esta política, porque conhece os outbounds configurados e consegue trocar de destino sem exigir alterações no cliente.

A decisão de colocar fallback na Gateway, e não nos clientes, reduz duplicação e melhora controlo operacional. Se cada aplicação cliente implementasse a sua própria lógica de fallback, seria necessário repetir regras de prioridade, tratamento de erros, mapeamento de contratos e métricas em vários pontos. Ao centralizar a política, o SDK consegue manter o mesmo contrato de entrada e decidir internamente que fornecedor ou modelo alternativo deve ser tentado. Isto

é particularmente útil quando o cliente usa um contrato compatível com Anthropic ou OpenAI, mas a Gateway pode executar a chamada noutro provider configurado.

A implementação atual usa a lista de outbounds disponíveis. Primeiro tenta o outbound correspondente ao fornecedor e modelo pedidos; se falhar, acrescenta esse outbound ao conjunto de negados e tenta alternativas. Para cada tentativa, cria um novo `GatewayContext` com os mesmos atributos, mas com `provider` e `model` atualizados para o outbound alternativo.

A integração com o circuit breaker é feita através do conjunto partilhado de outbounds negados. Assim, se um circuito estiver aberto, o fallback evita esse destino e tenta outro modelo ou fornecedor. Este desenho permite construir políticas de resiliência como “tentar Gemini, depois OpenAI-compatible via Groq, depois Anthropic”, mantendo a lógica de tentativa fora dos adapters. Também permite distinguir degradação controlada de erro total: a resposta pode chegar por um modelo alternativo, enquanto métricas e tracing registam que ocorreu fallback.

A evolução futura poderá acrescentar prioridades configuráveis, condições por código de erro, orçamentos de tentativa e métricas específicas de fallback. Estas regras são importantes porque nem todo erro justifica fallback: erros de autenticação ou autorização devem falhar imediatamente, enquanto timeouts, 429 e 5xx podem justificar uma alternativa.

5.6.7 Interceptor de Logging e Tracing

O `RequestLoggingInterceptor` regista fornecedor, modelo, sucesso, erro e conclusão de streams através da abstração `GatewayLogger`. No target JVM pode ser ligado a SLF4J; no target JavaScript pode usar console ou outra integração. O objetivo é garantir visibilidade mínima mesmo quando a telemetria completa não está ativa.

O logging foi mantido separado das métricas porque responde a perguntas diferentes. Métricas mostram tendências agregadas, como latência média ou número de erros; logs ajudam a explicar um pedido concreto. Numa Gateway de IA, esta distinção também tem impacto de privacidade: o objetivo é registar metadados operacionais, e não despejar prompts, respostas completas ou tokens sensíveis nos logs.

O `TracingInterceptor` envolve o pedido num *span* chamado `gateway.request.process`. Quando combinado com OpenTelemetry, este span permite correlacionar latência, erros, chamadas externas e métricas agregadas. Em cenários de fallback ou circuit breaker, tracing é particularmente útil porque permite observar o percurso real do pedido e distinguir falhas do fornecedor, decisões da Gateway e erros de cliente. Também permite perceber se uma resposta lenta resulta do fornecedor principal, de uma tentativa alternativa, de validação de token ou de uma política interna.

5.7 MCP Broker e Ferramentas Externas

O MCP Broker foi introduzido como consequência do estudo do MCP e do *tool calling*. O objetivo é separar a gestão de ferramentas externas da lógica específica das APIs de inferência. Em vez de cada adapter conhecer diretamente as ferramentas disponíveis e a forma como são executadas, o broker fornece uma camada intermédia para representar ferramentas, recursos e *prompts* de forma uniforme.

O módulo `mcp-broker` depende da biblioteca `io.modelcontextprotocol:kotlin-sdk`. A implementação atual inclui modelos para ferramentas, recursos e *prompts*, geração de esquemas JSON, configuração de transportes `stdio`, SSE e WebSocket, e mapeamento entre o domínio do projeto e tipos do SDK MCP. A classe `McpTool` descreve uma ferramenta por nome, descrição, serializador e função `execute`; `McpServerBuilder` permite registrar ferramentas, recursos e *prompts* de forma declarativa.

A Figura 5.7 representa o ciclo geral de *tool calling*. Primeiro, o cliente envia um pedido com ferramentas disponíveis. Em seguida, o modelo pode pedir uma chamada estruturada com nome e argumentos. A aplicação executa ou encaminha a ferramenta, devolve o resultado ao modelo e recebe a resposta final. O MCP Broker foi desenhado para suportar esta separação no futuro, permitindo que ferramentas sejam registradas e disponibilizadas sem acoplar cada adapter de inferência ao conjunto concreto de ferramentas.

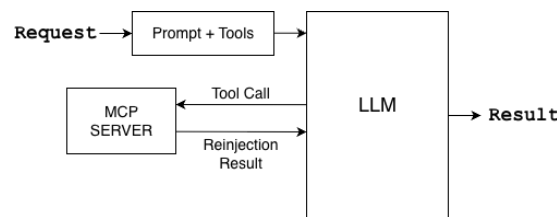


Figura 5.7: Representação do ciclo de *tool calling* e interação com ferramentas externas

No estado atual, o módulo estabelece sobretudo as abstrações, os modelos e o mapeamento necessários para essa evolução. A execução completa de ferramentas via MCP Broker, o carregamento por YAML e a integração no POC OmniAI Gateway permanecem como trabalho futuro. Esta delimitação é importante: o POC valida contratos HTTP, autenticação, telemetria e consumo do SDK, mas ainda não demonstra invocação real via MCP Broker.

5.8 Distribuição Multiplataforma

Um dos objetivos do OmniAI SDK é permitir que a lógica principal da Gateway seja reutilizada em diferentes ambientes de execução. O Kotlin Multiplatform permite separar código comum de implementações específicas: o domínio, contratos, portas, adapters principais, DSL e parte dos interceptors vivem em source sets comuns; detalhes como cliente HTTP, reflexão de prioridades, servidor Ktor e integrações de plataforma ficam em source sets específicos.

No target JVM, o OmniAI-SDK pode ser utilizado por aplicações Kotlin ou Java. Este é o target mais consolidado no estado atual do projeto e é o usado pelo POC OmniAI Gateway. A implementação JVM inclui Ktor HTTP client, integração com servidor Ktor, autenticação com descoberta OIDC, telemetria e execução real de outbounds. A distribuição futura via Maven permitirá que uma aplicação adicione o SDK como dependência versionada, sem depender de *composite build*.

No target JavaScript/Node.js, o projeto prepara uma API consumível a partir do ecossistema Node.js e TypeScript. A fachada `@JsExport` encapsula a configuração em tipos exportáveis e evita expor diretamente estruturas Kotlin que não são adequadas à fronteira JS. Esta camada é uma base para consumo via NPM, mas ainda não tem paridade total com a JVM.

A nuance principal da distribuição multiplataforma é que nem tudo deve ser comum. O domínio e os contratos devem permanecer partilhados; transporte HTTP, servidor, observabilidade e segurança podem ter implementações diferentes por plataforma. Esta divisão evita duplicação sem forçar uma abstração demasiado rígida. A linha de evolução prevista é consolidar a API pública do OmniAI-SDK, estabilizar versionamento semântico, automatizar publicação Maven/NPM por CD e documentar exemplos mínimos de consumo para JVM e Node.js.

Capítulo 6

Provas de Conceito (PoCs) e Validação

6.1 Validação do POC OmniAI Gateway

A validação principal foi realizada com a aplicação `OmniAiGateway`, um POC que consome o OmniAI SDK através do *composite build*. Esta aplicação carrega o ficheiro `application.conf`, resolve variáveis de ambiente, instancia o `KtorHttpClient`, configura outbounds, ativa segurança e telemetria, monta o runtime do SDK e expõe endpoints HTTP compatíveis com OpenAI, Anthropic e Gemini.

O comportamento validado não deve ser descrito como roteamento inteligente. A implementação atual realiza despacho configurado: o pedido contém fornecedor e modelo, e o dispatcher padrão escolhe o outbound que corresponde a essa combinação. Este desenho já permite demonstrar o desacoplamento entre cliente e fornecedor, mas políticas mais avançadas, como seleção por custo, qualidade, latência ou plano do cliente, permanecem como trabalho futuro. O trabalho de *fallback* e *circuit breaker* fornece a base técnica para esse tipo de evolução, mas o POC não afirma executar roteamento semântico ou otimizado.

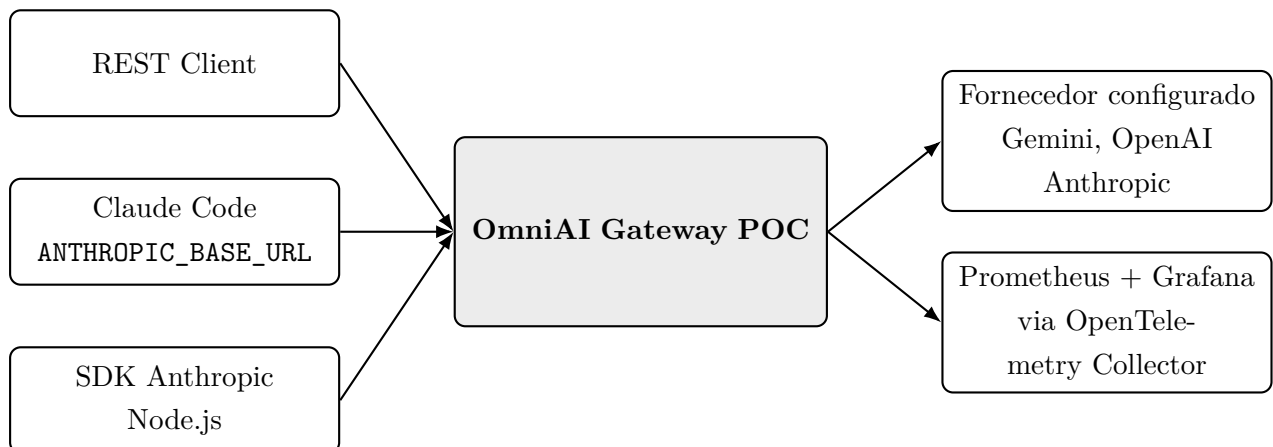


Figura 6.1: Cenários de validação do POC OmniAI Gateway

6.2 Validação com REST Client

O ficheiro `gateway-requests.http` foi usado para validar manualmente os endpoints expostos pela Gateway. Este ficheiro contém pedidos para `/v1/chat/completions`, `/v1/messages` e `/v1beta/models/{model}:generateContent`, cobrindo respostas unárias, *streaming*, geração com parâmetros e integração com token Bearer.

Este cenário foi importante por duas razões. Primeiro, permitiu validar a forma exata dos contratos HTTP recebidos pelo POC sem depender de uma aplicação externa complexa. Segundo, permitiu testar rapidamente o fluxo completo: token emitido pelo Logto, pedido autenticado, tradução inbound, despacho para outbound, chamada ao fornecedor configurado e resposta no contrato esperado pelo cliente.

6.3 Validação de Segurança com Logto

A camada de segurança foi validada com Logto como Authorization Server local. O ambiente Docker inclui Logto, PostgreSQL, OpenTelemetry Collector, Prometheus e Grafana. A configuração do POC usa OIDC Discovery para obter metadados do servidor de autorização e validar tokens com audiência configurada para o recurso da API.

Durante o desenvolvimento foram realizados testes exploratórios com Keycloak, mas a configuração final mudou para Logto por reduzir a complexidade operacional do POC. O Logto facilitou a criação de API Resources, aplicações Machine-to-Machine e fluxos OIDC adequados à validação da Gateway como Resource Server.

O fluxo também foi validado com `run_claude.py`. Este script abre o browser para autenticação no Logto, usa PKCE, troca o código por um token e inicia o Claude Code com `ANTHROPIC_BASE_URL` a apontar para a OmniAI Gateway. O cabeçalho `Authorization` é injetado através de `ANTHROPIC_CUSTOM_HEADERS`, permitindo testar um cliente real compatível com Anthropic a passar pela Gateway.

6.4 Validação de Observabilidade

A observabilidade foi validada com OpenTelemetry Collector, Prometheus e Grafana. O POC ativa métricas na DSL do SDK, incluindo latência padrão, contagem de tokens e contagem de erros. Também acrescenta atributos como IP do cliente, versão do SDK, audiência e dados derivados do resultado de autenticação, como `sub`, `email` ou `preferred_username` quando disponíveis.

O objetivo desta validação foi confirmar que a Gateway consegue produzir telemetria em tempo real enquanto processa pedidos reais. A utilização de Prometheus e Grafana permitiu observar métricas operacionais da aplicação e confirmar que a instrumentação está separada da lógica de inferência, sendo configurada através da DSL e exportada por adapters próprios.

6.5 Validação com Aplicações Cliente

Foram estudados e testados cenários com clientes que permitem alterar o endpoint da API. No caso do Claude Code, a variável `ANTHROPIC_BASE_URL` permite redirecionar pedidos para a Gateway.

Foi criado um teste com o sdk oficial da anthropic em `node-tests/anthropicSDKTest.js`, configurado com `baseUrl` local e chave fictícia, para validar que um cliente existente consegue falar com a OmniAI Gateway quando esta expõe um contrato compatível.

Também foram estudadas aplicações como OpenCode, LibreChat e Dify, por permitirem configurar endpoints compatíveis com fornecedores existentes. Estes estudos demonstram o valor arquitetural da Gateway: quando o cliente já segue um contrato conhecido, a introdução da OmniAI Gateway pode ser feita por configuração de endpoint, sem alterar a lógica principal da aplicação cliente.

6.6 Limites da Validação

A validação realizada confirma o consumo do SDK pelo POC, a exposição HTTP com Ktor, a integração com Logto, o uso de Prometheus/Grafana e a compatibilidade com clientes reais ou semi-reais. No entanto, existem limites importantes. O POC ainda não demonstra invocação de ferramentas via MCP Broker, nem políticas avançadas de roteamento por custo, latência ou qualidade. Estes pontos dependem de trabalho futuro sobre o módulo `mcp-broker`, sobre políticas de dispatcher e sobre stores distribuídos para cenários multi-instância.

Capítulo 7

Conclusões e Trabalho Futuro

7.1 Conclusões

O projeto evoluiu de uma Gateway específica para um SDK reutilizável capaz de instanciar gateways de IA configuráveis. Esta evolução tornou a solução mais alinhada com o problema identificado: reduzir acoplamento entre aplicações cliente e fornecedores de LLMs, centralizando contratos, autenticação, autorização, observabilidade, resiliência e integração com ferramentas.

O OmniAI SDK demonstra que é possível definir um domínio comum para pedidos, respostas, eventos de *streaming* e ferramentas, mantendo adapters específicos para OpenAI, Anthropic e Gemini. A Arquitetura Hexagonal ajudou a manter esta separação, enquanto a *pipeline* de *interceptors* permitiu adicionar comportamentos transversais sem contaminar o domínio.

A OmniAI Gateway, construída sobre o SDK, valida a proposta através de um POC real em Ktor, configurável por ficheiro e variáveis de ambiente, com suporte para Logto, telemetria via OpenTelemetry, Prometheus e Grafana, e integração com clientes compatíveis com Anthropic. A validação não cobre ainda invocação completa via MCP Broker nem roteamento otimizado por custo ou qualidade, mas demonstra a base arquitetural necessária para essa evolução.

7.2 Melhorias Futuras e Escalabilidade

Como trabalho futuro, existem várias linhas de evolução. A primeira é consolidar a distribuição pública do SDK via Maven e NPM, garantindo documentação de instalação, versionamento semântico e exemplos de consumo. A segunda é completar a paridade entre targets JVM e JavaScript, sobretudo na montagem de runtime e transporte HTTP.

Outra melhoria importante é aprofundar o MCP Broker, incluindo carregamento efetivo de ferramentas via YAML, execução completa dos handlers MCP e suporte robusto a SSE e WebSocket. Também será relevante melhorar as políticas de roteamento, permitindo decisões com base em custo, latência, disponibilidade, qualidade do modelo, plano do cliente e limites de utilização.

Por fim, a camada de segurança pode evoluir para políticas de autorização mais expressivas, integração com PDPs externos, introspeção real de tokens opacos e mapeamento mais fino entre claims, planos e quotas. Estas melhorias permitiriam aproximar a OmniAI Gateway de uma solução pronta para ambientes multi-tenant e produção.

Apêndice A

Histórico de Planeamento e Riscos

Durante a fase de planeamento foram identificados riscos técnicos e operacionais que continuam relevantes para a evolução do projeto.

A.1 Riscos

Durante o desenvolvimento do projeto foram identificados vários riscos técnicos e operacionais. Estes riscos influenciaram decisões de desenho, prioridades de implementação e estratégias de validação.

- **Curva de aprendizagem tecnológica:** O projeto exigiu a utilização de tecnologias e conceitos com elevada complexidade, nomeadamente Kotlin Multiplatform, arquitetura hexagonal, APIs de inferência de diferentes fornecedores, autenticação baseada em OIDC/OAuth2 e o Model Context Protocol. Esta diversidade tecnológica representou um risco inicial, porque aumentou o tempo necessário para compreender os contratos externos, configurar o ambiente de desenvolvimento e definir uma arquitetura estável. A mitigação passou pela realização de protótipos incrementais, pelo estudo isolado das APIs de OpenAI, Anthropic e Gemini, e pela separação progressiva do sistema em módulos com responsabilidades claras.
- **Instabilidade de serviços externos:** A Gateway depende de serviços de terceiros, como APIs de inferência, servidores de autorização e ferramentas externas. Estes serviços podem apresentar indisponibilidade temporária, alterações nos contratos, limites de utilização ou comportamentos diferentes dos documentados. Este risco poderia comprometer a validação da solução e tornar os testes dependentes de fatores externos. Para reduzir este impacto, foram usados testes locais, servidores simulados, mocks e abstrações sobre o transporte HTTP. Além disso, a arquitetura inclui mecanismos de resiliência, como *fallback* e *circuit breaker*, que permitem lidar melhor com falhas de fornecedores.
- **Custos de utilização:** A execução de testes com modelos pagos pode gerar custos inesperados, especialmente em cenários de *streaming*, chamadas repetidas ou testes de integração. Este risco é relevante porque a Gateway incentiva a experimentação com múltiplos fornecedores e modelos. A mitigação passou por limitar o número de chamadas reais, utilizar

fornecedores com quotas gratuitas sempre que possível, recorrer a testes locais e validar grande parte da lógica através de contratos, adapters e servidores simulados.

- **Complexidade de normalização entre fornecedores:** As APIs de OpenAI, Anthropic e Gemini partilham conceitos semelhantes, como mensagens, modelos, parâmetros de geração e ferramentas, mas representam esses conceitos de formas diferentes. Alguns elementos, como *tool calling*, *streaming*, multimodalidade e metadados específicos, não têm correspondência direta entre fornecedores. Este risco poderia levar a uma abstração demasiado rígida ou à perda de informação específica de cada API. Para o mitigar, foi definido um domínio comum extensível, complementado por campos de opções específicas do fornecedor. Desta forma, o SDK consegue representar o núcleo comum sem impedir a passagem de características particulares de cada API.
- **Segurança e dados sensíveis:** A centralização dos pedidos numa Gateway aumenta a responsabilidade sobre autenticação, autorização, gestão de tokens, logs, métricas e chaves de API. Um erro nesta camada poderia expor dados sensíveis ou permitir acesso indevido a modelos e recursos. A mitigação passou pela introdução de uma pipeline de segurança com autenticação e autorização separadas, validação de tokens, integração com um Authorization Server e cuidado na recolha de métricas e logs. Ainda assim, este risco continua relevante para uma evolução futura em direção a ambientes de produção e cenários multi-tenant.
- **Âmbito e complexidade funcional:** O domínio das Gateways de IA inclui várias funcionalidades possíveis, como roteamento por custo, políticas avançadas de autorização, execução automática de ferramentas, suporte completo a MCP, cache semântica e gestão de quotas por utilizador. Implementar todas estas capacidades numa primeira versão aumentaria significativamente o risco de atrasos e instabilidade. A mitigação passou pela definição de um âmbito inicial mais controlado, focado na arquitetura base, nos adapters principais, na pipeline de interceptors e numa Gateway funcional construída sobre o SDK. As funcionalidades mais avançadas foram identificadas como trabalho futuro.

A.2 Planeamento

Semana	Período	Descrição
1	18 fev – 22 fev (parcial)	Estudo das APIs de inferência e protocolo MCP.
2	23 fev – 1 mar	Continuação da semana anterior e início da proposta.
3	2 mar – 8 mar	Entrega Proposta do projeto
4	9 mar – 15 mar	Configuração inicial da biblioteca, definição da arquitetura base e das interfaces.
5	16 mar – 22 mar	Integração com as APIs de inferência. Foco inicial em dois fornecedores estáveis.
6	23 mar – 29 mar	Integração do MCP no OmniAI Core. Invocação e registo dinâmico de ferramentas, gerindo o ciclo de <i>tool calling</i> .
7	30 mar – 5 abr	Implementação de despacho configurável de modelos e mecanismos de redundância (<i>fallback</i> e <i>retry</i>).
8	6 abr – 12 abr	Implementação dos mecanismos de segurança, como controlo de acesso, preparação do contexto e remoção de dados sensíveis.
9	13 abr – 19 abr	Conclusão das implementações anteriores.
10	20 abr – 26 abr	Apresentação de Progresso
11	27 abr – 3 mai	Início do desenvolvimento da aplicação final OmniAI Gateway. Exposição da WEB API.
12	4 mai – 10 mai	Continuação da semana anterior e preparação das abstrações MCP para integração futura.
13	11 mai – 17 mai	Término de todas as APIs de comunicação para o exterior.
14	18 mai – 24 mai	Testes de integração do Core e da Gateway.
15	25 mai – 31 mai	Entrega da Versão beta
16	1 jun – 7 jun	Refinamento de testes a todos os módulos.
17	8 jun – 14 jun	Refinamento de código, desempenho e limpeza de bugs do sistema.
18	15 jun – 21 jun	Continuação da semana anterior e refinamento do relatório final.
19	22 jun – 28 jun 2026	Conclusão do relatório final.
20	29 jun – 5 jul 2026	Vistoria completa a todo o sistema e polimento de falhas para a entrega da Versão final.
21	6 jul – 11 jul 2026 (parcial)	Entrega Final do Projeto e Relatório

Tabela A.1: Planeamento semanal do projeto

Referências

- [1] IBM, “What are large language models (LLMs)?” [Online]. Available: <https://www.ibm.com/think/topics/large-language-models>
- [2] NVIDIA, “Ai tokens explained.” [Online]. Available: <https://blogs.nvidia.com/blog/ai-tokens-explained/>
- [3] OpenAI, “Openai api documentation.” [Online]. Available: <https://platform.openai.com/docs/>
- [4] Anthropic, “Anthropic messages api documentation.” [Online]. Available: <https://docs.anthropic.com/en/api/messages>
- [5] Google, “Gemini api documentation.” [Online]. Available: <https://ai.google.dev/docs>
- [6] Groq, “Groq api documentation.” [Online]. Available: <https://console.groq.com/docs>
- [7] IBM, “What are agentic workflows?” [Online]. Available: <https://www.ibm.com/think/topics/agentic-workflows>
- [8] Model Context Protocol, “Model context protocol specification.” [Online]. Available: <https://modelcontextprotocol.io>
- [9] A. Islam, K. Z. Zamli *et al.*, “An exploratory study of machine learning code smells.” [Online]. Available: <https://arxiv.org/abs/2103.04146>
- [10] N. van den Berg, “Undesired code duplication in machine learning pipelines.” [Online]. Available: <https://njfberg.medium.com/undesired-code-duplication-in-machine-learning-pipelines-d24dabcfa673>
- [11] S. P, “System design of api gateway.” [Online]. Available: <https://medium.com/@lazygeek78/system-design-of-api-gateway-e8924b71b7fb>
- [12] Vasanthan, “Ai gateway pattern: Centralized model access layer.” [Online]. Available: <https://medium.com/@vasanthancomrads/ai-gateway-pattern-centralized-model-access-layer-c5049e4f151f>
- [13] K. Stoney, “Building an ai gateway.” [Online]. Available: <https://karlstoney.com/building-an-ai-gateway/>
- [14] Berri AI, “LiteLLM: Call all llm apis using the openai format.” [Online]. Available: <https://www.litellm.ai/>

- [15] OpenRouter, “OpenRouter: A unified interface for llms.” [Online]. Available: <https://openrouter.ai/>
- [16] JetBrains, “Kotlin multiplatform documentation.” [Online]. Available: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/get-started.html>
- [17] A. D. Blog, “Hexagonal architecture: Understanding ports and adapters.” [Online]. Available: <https://medium.com/@akdevblog/hexagonal-architecture-understanding-ports-4020009e1aad>
- [18] Jakarta EE, “Servlet filter and filterchain api.” [Online]. Available: <https://jakarta.ee/specifications/servlet/>
- [19] Model Context Protocol, “Architecture overview.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>
- [20] —, “Kotlin sdk for model context protocol.” [Online]. Available: <https://github.com/modelcontextprotocol/kotlin-sdk>
- [21] A. Cockburn, “Hexagonal architecture.” [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture>
- [22] Gradle, “Composite builds.” [Online]. Available: https://docs.gradle.org/current/userguide/composite_builds.html
- [23] OpenTelemetry, “Opentelemetry protocol specification.” [Online]. Available: <https://opentelemetry.io/docs/specs/otlp/>
- [24] —, “What is opentelemetry?” [Online]. Available: <https://opentelemetry.io/docs/what-is-opentelemetry/>
- [25] D. Hardt, “RFC 6749: The oauth 2.0 authorization framework.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>
- [26] M. Jones and D. Hardt, “RFC 6750: The oauth 2.0 authorization framework: Bearer token usage.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6750>
- [27] OAuth.com, “The resource server.” [Online]. Available: <https://www.oauth.com/oauth2-servers/the-resource-server/>
- [28] M. Jones, N. Sakimura, and J. Bradley, “RFC 8414: Oauth 2.0 authorization server metadata.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8414>
- [29] M. Jones, J. Bradley, and N. Sakimura, “RFC 7519: Json web token (JWT).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [30] M. Jones, “RFC 7517: Json web key (JWK).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7517>
- [31] M. Jones, J. Bradley, and N. Sakimura, “RFC 7515: Json web signature (JWS).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7515>

- [32] J. Richer, “RFC 7662: OAuth 2.0 token introspection.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7662>
- [33] Logto, “Logto documentation.” [Online]. Available: <https://docs.logto.io/>
- [34] Keycloak, “Keycloak documentation.” [Online]. Available: <https://www.keycloak.org/documentation>
- [35] Microsoft, “Authentication vs. authorization.” [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity-platform/authentication-vs-authorization>
- [36] Identity Beyond Borders, “The four horsemen of authorization: The p’s, pep, pdp, pap, and pip.” [Online]. Available: <https://medium.com/identity-beyond-borders/the-four-horsemen-of-authorization-the-p-ps-pep-pdp-pap-and-pip-42717e445ce7>
- [37] Cloudflare, “O que é limitação de taxa?” [Online]. Available: <https://www.cloudflare.com/pt-br/learning/bots/what-is-rate-limiting/>
- [38] Microsoft, “Circuit breaker pattern.” [Online]. Available: <https://learn.microsoft.com/azure/architecture/patterns/circuit-breaker>
- [39] Itau Tecnologia, “Compreendendo o conceito de circuit breaker e sua aplicação em arquitetura de microsserviços.” [Online]. Available: <https://medium.com/@itautecnologia/compreendendo-o-conceito-de-circuit-breaker-e-sua-aplicacao-em-arquitetura-de-microsservicos-108dc2d1be2c>
- [40] TreinaWeb, “O que é circuit breaker?” [Online]. Available: <https://www.treinaweb.com.br/blog/o-que-e-circuit-breaker>
- [41] T. Tombas, “Building resilient ai systems: Understanding model-level fallback mechanisms.” [Online]. Available: <https://medium.com/@tombastaner/building-resilient-ai-systems-understanding-model-level-fallback-mechanisms-436cf636045f>
- [42] Portkey, “Fallbacks.” [Online]. Available: <https://portkey.ai/docs/product/ai-gateway/fallbacks>